



Addis Ababa University
College of Natural and Computational Sciences
Department of Computer Science

Programming and Algorithms Module

Part I: Computer Programming

Part II: Object Oriented Programming

 **Part III: Design and Analysis of Algorithms**

Part IV: Data Structure and Algorithms

May 2024
Addis Ababa,
Ethiopia

Chapter One

Introduction to Design and analysis of algorithms

Chapter Objectives: At the end of this chapter you should be able to:

- Describe how to analyze a program.
- Explain how to choose the operations that are counted and why others are not.
- Explain how to do a best-case, worst-case and average-case analysis.
- Work with logarithms, probabilities and summations.
- Describe $\Theta(f)$, $\Omega(f)$, $O(f)$, growth rate, and algorithm order.
- Use decision tree to determine a lower bound on complexity
- Convert a simple recurrence relation into its closed form.

1.1 Introduction to Data Structures and Algorithms Analysis

A program is written in order to solve a problem. A solution to a problem actually consists of two things:

- A way to organize the data
- Sequence of steps to solve the problem

The way data are organized in a computer's memory is said to be Data Structure and the sequence of computational steps to solve a problem is said to be an algorithm. Therefore, a program is nothing but data structures plus algorithms.

1.2 What is an algorithms

An algorithm is a well-defined computational procedure that takes some value or a set of values as input and produces some value or a set of values as output. Data structures model the static part of the world. They are unchanging while the world is changing. In order to model the dynamic part of the world we need to work with algorithms. Algorithms are the dynamic part of a program's world model.

An algorithm transforms data structures from one state to another state in two ways:

- An algorithm may change the value held by a data structure
- An algorithm may change the data structure itself

The quality of a data structure is related to its ability to successfully model the characteristics of the world. Similarly, the quality of an algorithm is related to its ability to successfully simulate the changes in the world.

However, independent of any particular world model, the quality of data structure and algorithms is determined by their ability to work together well. Generally speaking, correct data structures lead to simple and efficient algorithms and correct algorithms lead to accurate and efficient data structures.

Properties of an algorithm

- ***Finiteness***: Algorithm must complete after a finite number of steps.
- ***Definiteness***: Each step must be clearly defined, having one and only one interpretation. At each point in computation, one should be able to tell exactly what happens next.
- ***Sequence***: Each step must have a unique defined preceding and succeeding step. The first step (start step) and last step (halt step) must be clearly noted.
- ***Feasibility***: It must be possible to perform each instruction.
- ***Correctness***: It must compute correct answer for all possible legal inputs.
- ***Language Independence***: It must not depend on any one programming language.
- ***Completeness***: It must solve the problem completely.
- ***Effectiveness***: It must be possible to perform each step exactly and in a finite amount of time.
- ***Efficiency***: It must solve with the least amount of computational resources such as time and space.
- ***Generality***: Algorithm should be valid on all possible inputs
- ***Input/Output***: There must be a specified number of input values, and one or more result values.

1.3. Complexity analysis

1.3.1. Space and Time Complexity

Performance Analysis

By performance of a program, we mean the amount of computer memory and time needed to run a program.

There are two approaches to determine the performance of a program:

- Analytical approach- used for performance analysis,
- Experimental approach – conduct experiments for performance measurement.

What is Analysis: - The analysis of algorithm provides background information that a general idea of how long an algorithm will take for a given set. For each algorithm considered, we will come up with an estimate of how long it will take to solve a problem that has a set of N input values. For example:-

i. We might determine how many comparisons a sorting algorithm does to put a list of N values in to

ascending order, or,

- ii. We might determine how many arithmetic operations it takes to multiply two matrices of size $N \times N$

There are a number of algorithms that will solve a problem. Studying the analysis of algorithms gives us the tools to choose between algorithms.

The complexity of an algorithm can be expressed in terms of its space and time complexity.

Space Complexity of a program: - is the amount of memory space it needs to run to completion. We are interested in the space complexity of a program for the following reasons:

- If a program is to run on a multiuser computer system, then we may need to specify the amount of memory to be allocated for the program.
- For any computer system, we would like to know in advance whether or not sufficient memory is available to run the program.
- A problem might have several possible solutions with different space requirements.
- We can use the space complexity to estimate the size of the largest problem that a program can solve. For example, we may have a circuit simulation program that requires $10^6 + 100(c + w)$ bytes of memory to simulate circuits with c components and w wires. If the total amount of memory available is $4 * 10^6$, then we can simulate circuits with $c + w \leq 30,000$.

Components of space complexity: - the space needed by a program has the following components:

- **Instruction space:** - is the amount of space needed to store the compiled version of the program instructions. The amount of instruction space that is needed depends on factors such as:
 - The compiler used to compile the program into machine code.
 - The compiler options in effect at time of compilation.
 - The target computer.
- **Data space:** - is the space needed to store all constant and variable values. Data space has two components:
 - i. Space needed by constants (e.g. the numbers 0, 1, 2...) and simple variables.
 - ii. Space needed by dynamically allocated objects such as arrays and class instances.
- **Environmental stack space:** - is the space used to save information needed to resume execution of partially completed methods (functions). For example, if method **func1** invokes or calls method **func2**, then we must at least save a pointer to the instruction of **func1** to be executed when **func2** terminates. Each time a function or method is called or invoked, the following data are stored on the

environment stack.

- i. The return address

- ii. The values of all local variables and formal parameters in the method or function being invoked or called.

Time Complexity of a program: - is the amount of computer time it needs to run to completion. We are interested in time complexity of program for the following reasons:

- Some computer systems require the user to provide an upper limit on the amount of time the program will run. Once this upper limit is reached, the program is aborted. An easy way out is to simply specify a time limit of a few thousand years. However, this solution could result in serious fiscal problems if the program runs into an infinite loop caused by some discrepancy in the data and you actually get billed for the computer time used. We would like to provide a time limit that is just slightly above the expected run time.
- The program we are developing might need to provide a satisfactory real-time response. For example, all interactive programs must provide such a response. A text editor that takes a minute to move the cursor one page down or one page up will not be acceptable to many users. A spreadsheet program that takes several minutes to re-evaluate the cells in a sheet will be satisfactory only to patient users. A database management system that allows its users adequate time to drink two cups of coffee while it is sorting a relation will not find too much acceptance. Programs designed for interactive use must provide satisfactory real-time response. From the time complexity of the program or program module, we can decide whether or not the response time will be acceptable. If not, we need to either redesign the algorithm or give the user a faster computer.

If we have alternative ways to solve a problem, then the decision on which to use will be based primarily on the expected performance difference among those solutions. We will use some weighted measures of the space and time complexities of the alternative solutions.

Components of time complexity: - the time complexity of a program depends on all the factors that the space complexity depends on.

Since analytical approach to determine the run time of a program is fraught with complications, we attempt only to estimate the run time. To more manageable approach to estimating the run time are:

- i. Identify one or more key operations and determine the number of times these are performed, and
- ii. Determine the total number of steps executed by the program.

1.4. Algorithm Analysis

Complexity analysis involves two distinct phases:

Algorithm Analysis: Analysis of the algorithm or data structure to produce a function $T(n)$ that describes the

algorithm in terms of the operations performed in order to measure the complexity of the algorithm.

- **Order of Magnitude Analysis:** Analysis of the function $T(n)$ to determine the general complexity category to which it belongs.

There is no generally accepted set of rules for algorithm analysis. However, an exact count of operations is commonly used.

Analysis Rules:

1. We assume an arbitrary time unit.
2. Execution of one of the following operations takes time 1:
 - Assignment Operation
 - Single Input/output Operation
 - Single Boolean Operations
 - Single Arithmetic Operations
 - Function Return
3. Running time of a selection statement (if, switch) is the time for the condition evaluation plus the maximum of the running times for the individual clauses in the selection.
4. **Loops:** Running time for a loop is equal to the running time for the statements inside the loop multiplied by the number of iterations.

The total running time of a statement inside a group of nested loops is the running time of the statements multiplied by the product of the sizes of all the loops.

For nested loops, analyze inside out.

- Always assume that the loop executes the maximum number of iterations possible.
5. Running time of a function call is 1 for setup plus the time for any parameter calculations plus the time required for the execution of the function body.

Examples

Example1:

```
int count()
{
    int k=0;

    cout<< "Enter an integer";

    cin>>n;

    for (i=0;i<n;i++)

        k=k+1;
```

Time Units to Compute

1 for the assignment statement: `int k=0`
 1 for the output statement.
 1 for the input statement.
 In the for loop:
 1 assignment, $n+1$ tests, and n increments.
 n loops of 2 units for an assignment, and an addition.
 1 for the return statement.

$$T(n) = 1 + 1 + 1 + (1 + n + 1 + n) + 2n + 1 = 4n + 6 = O(n)$$

```
    return 0;  
}
```

Formal Approach to Analysis

In the above examples we have seen that analysis is a bit complex. However, it can be simplified by using some formal approach in which case we can ignore initializations, loop control, and book keeping.

for Loops: Formally

- In general, a for loop translates to a summation. The index and bounds of the summation are the same as the index and bounds of the for loop.

```
for (int i = 1; i <= N; i++) {  
    sum = sum+i;  
}
```

$$\sum_{i=1}^N 1 = N$$

- Suppose we count the number of additions that are done. There is 1 addition per iteration of the loop, hence N additions in total.

Nested Loops: Formally

- Nested for loops translate into multiple summations, one for each for loop.

```
for (int i = 1; i <= N; i++) {  
    for (int j = 1; j <= M; j++) {  
        sum = sum+i+j;  
    }  
}
```

$$\sum_{i=1}^N \sum_{j=1}^M 2 = \sum_{i=1}^N 2M = 2MN$$

- Again, count the number of additions. The outer summation is for the outer for loop.

Consecutive Statements: Formally

- Add the running times of the separate blocks of your code

```
for (int i = 1; i <= N; i++) {  
    sum = sum+i;  
}  
for (int i = 1; i <= N; i++) {  
    for (int j = 1; j <= N; j++) {  
        sum = sum+i+j;  
    }  
}
```

$$\left[\sum_{i=1}^N 1 \right] + \left[\sum_{i=1}^N \sum_{j=1}^N 2 \right] = N + 2N^2$$

Conditionals: Formally

- If (test) s1 else s2: Compute the maximum of the running time for s1 and s2.

```
if (test == 1) {  
    for (int i = 1; i <= N; i++) {  
        sum = sum+i;  
    }  
} else for (int i = 1; i <= N; i++) {  
    for (int j = 1; j <= N; j++) {  
        sum = sum+i+j;  
    }  
}
```

$$\max \left(\sum_{i=1}^N 1, \sum_{i=1}^N \sum_{j=1}^N 2 \right) =$$

$$\max(N, 2N^2) = 2N^2$$

1.5. Measures of Times

In order to determine the running time of an algorithm it is possible to define three functions $T_{best}(n)$, $T_{avg}(n)$ and $T_{worst}(n)$ as the best, the average and the worst case running time of the algorithm respectively.

Average Case (T_{avg}): The amount of time the algorithm takes on an "average" set of inputs.

Worst Case (T_{worst}): The amount of time the algorithm takes on the worst possible set of inputs.

Best Case (T_{best}): The amount of time the algorithm takes on the smallest possible set of inputs.

We are interested in the worst-case time, since it provides a bound for all input – this is called the “Big-Oh” estimate.

1.6. Asymptotic Analysis

In analysis of algorithms, it is not important to know how many operations an algorithm does. Of greater concern is the rate of increase in operations for an algorithm to solve a problem as the size of the problem increases. This is referred to as the **rate of growth of the algorithm**. What happens with small sets of input data is not as interesting as what happens when the data set gets large.

The rate of growth of an algorithm is dominated by the largest term in an equation; we will discard the terms that grow more slowly. When we strip all of these things away, we are left with what we call the **order** of the function or related algorithm. We then group algorithms based on their order. We group in three categories – those that grow at least as fast as some function, those that grow at the same rate, and those that grow no faster. The following are some of the classes of algorithms: - $\log_2 n$, n , $n * \log_2 n$, n^2, n^3 , 2^n .

Asymptotic analysis is concerned with how the running time of an algorithm increases with the size of the input in the limit, as the size of the input increases without bound.

There are five notations used to describe a running time function. These are:

- Big-Oh Notation (O)
- Big-Omega Notation (Ω)
- Theta Notation (Θ)
- Little-o Notation (o)
- Little-Omega Notation (ω)

1.7. Recurrences, Solution of Recurrences by substitution, Recursion Tree and Master Method

Recursion is a particularly powerful kind of reduction, which can be described loosely as follows:

- If the given instance of the problem is small or simple enough, just solve it.
- Otherwise, reduce the problem to one or more simpler instances of the same problem.

Recursion is generally expressed in terms of recurrences. In other words, when an algorithm calls to itself, we can often describe its running time by a recurrence equation which describes the overall running time of a problem of size n in terms of the running time on smaller inputs.

E.g. the worst case running time $T(n)$ of the merge sort procedure by recurrence can be expressed as

$$T(n) = \Theta(1) ; \text{ if } n=1$$

$$2T(n/2) + \Theta(n) ; \text{ if } n>1$$

whose solution can be found as $T(n) = \Theta(n \log n)$

There are various techniques to solve recurrences.

1.7.1. SUBSTITUTION METHOD:

The substitution method comprises of 3 steps

- i. Guess the form of the solution
- ii. Verify by induction
- iii. Solve for constants

We substitute the guessed solution for the function when applying the inductive hypothesis to smaller values. Hence the name “substitution method”. This method is powerful, but we must be able to guess the form of the answer in order to apply it.

Chapter Two

Design and analysis of Divide and Conquer Algorithms

2.1 Divide and Conquer Algorithm

In this approach, we solve a problem recursively by applying 3 steps

1. **DIVIDE**-break the problem into several sub problems of smaller size.
2. **CONQUER**-solve the problem recursively.
3. **COMBINE**-combine these solutions to create a solution to the original problem.

CONTROL ABSTRACTION FOR DIVIDE AND CONQUER ALGORITHM

Algorithm D and C (P)

```
{  
if small(P)  
then return S(P)  
else  
{ divide P into smaller instances P1 ,P2 ..... Pk  
Apply D and C to each sub problem  
Return combine (D and C(P1)+ D and C(P2)+.....+D and C(Pk))  
}}
```

Let a recurrence relation is expressed as	$T(n) = \Theta(1), \text{ if } n \leq C$ $aT(n/b) + D(n) + C(n), \text{ otherwise}$
---	--

then n =input size a =no. Of sub-problems n/b = input size of the sub-problems

- Breaking large problems into smaller subproblems
- Example of algorithms uses divide-and-conquer strategy :
 - ❖ Binary search,
 - ❖ Finding maximum and minimum
 - ❖ Merge sort
 - ❖ Quick sort
 - ❖ selection sort

2.2. BINARY SEARCH

Algorithm Bin search(a,n,x)

// Given an array a[1:n] of elements in non-decreasing

```
//order, n>=0,determine whether 'x' is present and
// if so, return 'j' such that x=a[j]; else return 0.
{
low:=1; high:=n;
while (low<=high) do {
mid:=[(low+high)/2];
if (x<a[mid]) then high;
else if(x>a[mid]) then
low=mid+1;
else return mid;
} return 0; }
```

- Algorithm, describes this binary search method, where Binsrch has 4I/ps a[], I, l & x.
- It is initially invoked as Binsrch (a,1,n,x)
- A non-recursive version of Binsrch is given below.
- This Binsearch has 3 i/ps a,n, & x.
- The while loop continues processing as long as there are more elements left to check.
- At the conclusion of the procedure 0 is returned if x is not present, or 'j' is returned, such that a[j]=x.
- We observe that low & high are integer Variables such that each time through the loop either x is found or low is increased by at least one or high is decreased at least one.
- Thus we have 2 sequences of integers approaching each other and eventually low becomes > than high & causes termination in a finite no. of steps if 'x' is not present.

2.4. Maximum and Minimum:

- Let us consider another simple problem that can be solved by the divide-and conquer technique.
- The problem is to find the maximum and minimum items in a set of 'n' elements.
- In analyzing the time complexity of this algorithm, we once again concentrate on the no. of element comparisons.
- More importantly, when the elements in a[1:n] are polynomials, vectors, very large numbers, or strings of character, the cost of an element comparison is much higher than the cost of the other operations.
- Hence, the time is determined mainly by the total cost of the element comparison.

Algorithm straight MaxMin(a,n,max,min)

// set max to the maximum & min to the minimum of a[1:n]

```
{  
max:=min:=a[1];  
for I:=2 to n do
```

```
{
if(a[I]>max) then max:=a[I];
if(a[I]<min) then min:=a[I];
} }
```

- Now the best case occurs when the elements are in increasing order.

◇ The no. of element comparison is $(n-1)$.

- The worst case occurs when the elements are in decreasing order.

◇ The no. of elements comparison is $2(n-1)$

- □ The average no. of element comparison is $<$ than $2(n-1)$
- □ On the average $a[I]$ is $>$ than max half the time, and so, the avg. no. of comparison is $3n/2-1$.
- If $T(n)$ represents this no., then the resulting recurrence relations is

$T(n) = \begin{cases} T([n/2]) + T([n/2]) + 2 & n > 2 \\ 1 & n = 2 \\ 0 & n = 1 \end{cases}$

1 $n=2$

0 $n=1$

*Note that $(3n/3)-3$ is the best-average, and worst-case no. of comparisons when 'n' is a power of 2.

MERGE SORT

- As another example divide-and-conquer, we investigate a sorting algorithm that has the nice property that in the worst case its complexity is $O(n \log n)$
- This algorithm is called merge sort
- We assume throughout that the elements are to be sorted in non-decreasing order.
- Given a sequence of 'n' elements $a[1], \dots, a[n]$ the general idea is to imagine then split into 2 sets $a[1], \dots, a[n/2]$ and $a[[n/2]+1], \dots, a[n]$.
- Each set is individually sorted, and the resulting sorted sequences are merged to produce a single sorted sequence of 'n' elements.
- Thus, we have another ideal example of the divide-and-conquer strategy in which the splitting is into 2 equal-sized sets & the combining operation is the merging of 2 sorted sets into one.

Algorithm For Merge Sort:

Algorithm MergeSort(low,high)

//a[low:high] is a global array to be sorted

//Small(P) is true if there is only one element
//to sort. In this case the list is already sorted.

```

{
if (low<high) then //if there are more than one element
{
//Divide P into subproblems
//find where to split the set
mid = [(low+high)/2];
//solve the subproblems.
mergesort (low,mid);
mergesort(mid+1,high);
//combine the solutions .
merge(low,mid,high);
} }

```

- If the time for the merging operations is proportional to 'n', then the computing time for merge sort is described by the recurrence relation.

$T(n) = \begin{cases} a & n=1, \\ a' & \text{a constant} \end{cases}$

$2T(n/2)+cn \quad n>1, 'c' \text{ a constant.}$

- Therefore, **$T(n)=O(n \log n)$**

QUICK SORT

- The divide-and-conquer approach can be used to arrive at an efficient sorting method different from merge sort.
- In merge sort, the file $a[1:n]$ was divided at its midpoint into sub arrays which were independently sorted & later merged.
- In Quick sort, the division into 2 sub arrays is made so that the sorted sub arrays do not need to be merged later.
- This is accomplished by rearranging the elements in $a[1:n]$ such that $a[i] \leq a[j]$ for all i between 1 & n and all j between $(m+1)$ & n for some m , $1 \leq m \leq n$.
- Thus the elements in $a[1:m]$ & $a[m+1:n]$ can be independently sorted.
- No merge is needed. This rearranging is referred to as partitioning.
- Function partition of Algorithm accomplishes an in-place partitioning of the elements of $a[m:p-1]$
- It is assumed that $a[p] \geq a[m]$ and that $a[m]$ is the partitioning element. If $m=1$ & $p-1=n$, then $a[n+1]$ must be defined and must be greater than or equal to all elements in $a[1:n]$
- The assumption that $a[m]$ is the partition element is merely for convenience, other

choices for the partitioning element than the first item in the set are better in practice.

- The function `interchange(a,i,j)` exchanges `a[i]` with `a[j]`.

Algorithm: Partition the array $a[m:p-1]$ about $a[m]$

Algorithm Partition(a, m, p)

//within $a[m], a[m+1], \dots, a[p-1]$ the elements

// are rearranged in such a manner that if //initially $t=a[m]$, then after completion

// $a[q]=t$ for some q between m and

// $p-1, a[k] \leq t$ for $m \leq k < q$, and $a[k] > t$ for $q < k < p$. q is returned //Set $a[p]=\text{infinite}$.

{

$v=a[m]; I=m; j=p;$

repeat

{

repeat

$I=I+1;$

until($a[I] \geq v$);

repeat

$j=j-1;$

until($a[j] \leq v$);

if ($I < j$) then interchange(a, i, j);

}until($I \geq j$);

$a[m]=a[j]; a[j]=v;$

return j ;

}

STRASSON'S MATRIX MULTIPLICATION

- Let A and B be the $2 \times n$ Matrix. The product matrix $C=AB$ is calculated by using the formula, $C(i, j) = A(i, k) B(k, j)$ for all 'i' and j between 1 and n .
- The time complexity for the matrix Multiplication is $O(n^3)$.
- Divide and conquer method suggest another way to compute the product of $n \times n$ matrix.
- We assume that N is a power of 2. In the case N is not a power of 2, then enough rows and columns of zero can be added to both A and B . So that the resulting dimension are the powers of two.
- If $n=2$ then the following formula as a computed using a matrix multiplication operation for the elements of A & B .
- If $n > 2$, Then the elements are partitioned into sub matrix $n/2 \times n/2$..since 'n' is a power of 2 these product can be recursively computed using the same formula. This Algorithm will continue applying itself to smaller sub matrix until 'N' become suitable small ($n=2$) so that the product is computed directly.

- The C_{ij} are required addition 8 addition or subtraction.

$T(n) = b \quad n \leq 2$ a & b are

$7T(n/2) + an^2$ $n > 2$ constant

Finally we get $T(n) = O(n^{\log_2 7})$.

UNIT – Three

Introduction to Greedy Model

GREEDY METHOD

- Greedy method is the most straightforward designed technique.
- As the name suggest they are short sighted in their approach taking decision on the basis of the information immediately at the hand without worrying about the effect these decisions may have in the future.

DEFINITION:

- A problem with N inputs will have some constraints. Any subsets that satisfy these constraints are called a feasible solution.
- A feasible solution that either maximize can minimize a given objectives function is called an optimal solution.

Greedy is a strategy that works well on optimization problems with the following characteristics:

- Greedy-choice property: A global optimum can be arrived at by selecting a local

optimum.

- Optimal substructure: An optimal solution to the problem contains an optimal solution to sub problems.

The second property may make greedy algorithms look like dynamic programming. However, the two techniques are quite different.

Control algorithm for Greedy Method:

Algorithm Greedy (a,n)

```
//a[1:n] contain the n inputs {  
  solution =0;//Initialise the solution.  
  For i=1 to n do  
    {  
      x=select(a);  
      if(feasible(solution,x))then  
        solution=union(solution,x);  
    } return solution;  
}
```

The function select an input from $a[]$ and removes it. The select input value is assigned to X .

- ✓ Feasible is a Boolean value function that determines whether X can be included into the solution vector.
- ✓ The function Union combines X with the solution and updates the objective function.
- ✓ The function Greedy describes the essential way that a greedy algorithm will once a particular problem is chosen, the function subset, feasible & union are properly implemented.

Greedy algorithms sometimes fail to produce the optimal solution, and may even produce the unique worst possible solution. One example is the travelling salesman problem.

Greedy algorithms can be characterized as being 'short sighted', and as 'non-recoverable'. They are ideal only for problems which have 'optimal substructure'. Despite this, greedy algorithms are best suited for simple problems (e.g. giving change). It is important, however, to note that the greedy algorithm can be used as a selection algorithm to prioritize options within a search, or branch and bound algorithm. There are a few variations to the greedy algorithm:

1. Pure greedy algorithms
2. Orthogonal greedy algorithms
3. Relaxed greedy algorithms

Greedy algorithms mostly (but not always) fail to find the globally optimal solution, because they usually do not operate exhaustively on all the data. They can make commitments to certain choices too early which prevent them from finding the best overall solution later. For example, all known greedy coloring algorithms for the graph coloring problem and all other NP-complete problems do not consistently find optimum solutions. Nevertheless, they are useful because they are quick to think up and often give good approximations to the optimum.

○ **Knapsack Problem**

- ✓ We are given n objects and knapsack or bag with capacity M . Object ' i ' has a weight W_i and profit P_i where i varies from 1 to N .
- ✓ The problem is we have to fill the bag with the help of N objects and the resulting profit has to be maximum.
- ✓ Formally the problem can be stated as

$$\text{Maximize } \sum_{1 \leq i \leq n} X_i P_i \quad \text{subject to } X_i W_i \leq M$$

Where X_i is the fraction of object and it lies between 0 to 1.

- There are so many ways to solve this problem, which will give many feasible solutions for which we have to find the optimal solution.

- But in this algorithm, it will generate only one solution which is going to be feasible as well as optimal.
- First, we find the profit & weight rates of each and every object and sort it according to the descending order of the ratios.
- Select an object with highest p/w ratio and check whether its height is lesser than the capacity of the bag.
- If so place 1 unit of the first object and decrement. the capacity of the bag by the weight of the object you have placed.
- Repeat the above steps until the capacity of the bag becomes less than the weight of the object you have selected .in this case place a fraction of the object and come out of the loop.
- Whenever you selected.

$$\text{maximize } \sum_{1 \leq i \leq n} p_i x_i \quad (4.1)$$

$$\text{subject to } \sum_{1 \leq i \leq n} w_i x_i \leq M \quad (4.2)$$

$$\text{and } 0 \leq x_i \leq 1, \quad 1 \leq i \leq n \quad (4.3)$$

The Profits and Weights are positive.

ALGORITHM:

Algorithm Greedy knapsack (m,n)

//P[1:n] and the w[1:n]contain the profit

• & weight res'.of the n object ordered.

//such that $p[i]/w[i] \geq p[i+1]/w[i+1]$

//n is the Knapsack size and x[1:n] is the solution vertex.

```
{
for i=1 to n do a[i]=0.0;
U=n;
for i=1 to n do
{
if (w[i]>u)then break;
x[i]=1.0;U=U-w[i]
}if(i<=n)then x[i]=U/w[i];
}
```

Example:

Capacity=20

N=3 ,M=20 Wi=18,15,10

Pi=25,24,15

$P_i/W_i=$
 $25/18=$
 1.36,
 $24/15=$
 1.6,
 $15/10=$
 1.5

Descending Order $\rightarrow P_i/W_i \rightarrow 1.6 \quad 1.5 \quad 1.36$

$P_i = 24 \quad 15 \quad 25$

$W_i = 15 \quad 10 \quad 18$

$X_i = 1$	5/10	0	$\frac{1}{2}$	$\frac{1}{3}$	$\frac{1}{4}$	16.6	24.25
$P_i * X_i = 1 * 24 + 0.5 * 15 = 31.5$			1	$\frac{2}{5}$	0	20	18.2
The optimal solution is $\rightarrow 31.5$			0	$\frac{2}{3}$	1	20	31
X_1	X_2	X_3	W_{IXI}	P_{IXI}			
			0	1	$\frac{1}{2}$	20	31.5

Of these feasible solutions Solution No 4 yields the Max profit. And this solution is optimal for the given problem instance.

Job Sequencing with Deadlines

The problem is the number of jobs, their profit and deadlines will be given and we have to find a sequence of job, which will be completed within its deadlines, and it should yield a maximum profit.

Points To remember:

- ☐ To complete a job, one has to process the job or a action for one unit of time.
- ☐ Only one machine is available for processing jobs.
- ☐ A feasible solution for this problem is a subset of j of jobs such that each job in this subject can be completed by this deadline.
- ☐ If we select a job at that time,
 - Since one job can be processed in a single m/c. The other job has to be in its waiting state until the job is completed and the machine becomes free.
 - So the waiting time and the processing time should be less than or equal to the dead line of the job.

ALGORITHM:

Algorithm JS(d,j,n) {

//The jobs are ordered such that $p[1] > p[2] \dots > p[n]$

//j[i] is the ith job in the optimal solution

✓ Also at terminal $d[j[i]] \leq d[j[i+1]], 1 < i < k$

$d[0] = j[0] = 0;$

$j[1] = 1; K = 1;$

for $I = 1$ to n do

{ // consider jobs in non increasing order of $P[I]$; find the position for I and check feasibility

insertion

$r = k;$

```

while((d[J[r]]>d[i])an
d(d[J[r]] = r)do r =r-1;
if (d[J[r]]<d[I])and (d[I]>r))then
{ for q=k to (r+1) step -1 do J [q+1]=j[q] J[r+1]=i;
K=k+1;
}} return k;
}

```

Example : 1. $n=5$ $(P1, P2, \dots, P5) = (20, 15, 10, 5, 1,)$
 $(d_1, d_2, \dots, d_5) = (2, 2, 1, 3, 3)$

Feasible Solution	Processing sequence	Value
(1)	(1)	20
(2)	(2)	15
(3)	(3)	10
(4)	(4)	5
(5)	(5)	1
(1,2)	(2,1)	35
(1,3)	(3,1)	30
(1,4)	(1,4)	25
(1,5)	(1,5)	21
(2,3)	(3,2)	25
(2,4)	(2,4)	20
(2,5)	(2,5)	16
(1,2,3)	(3,2,1)	45
(1,2,4)	(1,2,4)	40
The Solution 13 is optimal		

2. $n=4$ $(p1, p2, p3, p4) = (100, 10, 15, 27)$
 $(d_1, d_2, d_3, d_4) = (2, 1, 2, 1)$

<i>Feasible solution</i>		<i>Processing Sequence</i>	<i>Value</i>
(1,2)		(2,1)	110
(1,3)		(1,3)	115
(1,4)		(4,1)	127
(2,3)		(9,3)	25
(2,4)		(4,2)	37
(3,4)		(4,3)	42
(1)		(1)	100
(2)		(2)	10
(3)		(3)	15
(4)		(4)	27

The solution 3 is optimal.

○ **Minimum Cost Spanning Tree- Prim's Algorithm**

- ✓ Let $G(V,E)$ be an undirected connected graph with vertices v and edge E .
- ✓ A sub-graph $t=(V,E)$ of the G is a Spanning tree of G iff t is a tree.³
- ✓ The problem is to generate a graph $G = (V,E)$ where E is the subset of E,G is a Minimum spanning tree.
- ✓ Each and every edge will contain the given non-negative length. connect all the nodes with edge present in set E and weight has to be minimum.

NOTE:

- ✓ We have to visit all the nodes.
- ✓ The subset tree (i.e) any connected graph with N vertices must have at least $N-1$ edges and also it does not form a cycle.

Definition:

1. A spanning tree of a graph is an undirected tree consisting of only those edge that are necessary to connect all the vertices in the original graph.
2. A Spanning tree has a property that for any pair of vertices there exist only one path between them and the insertion of an edge to a spanning tree form a unique cycle.

Application of the spanning tree:

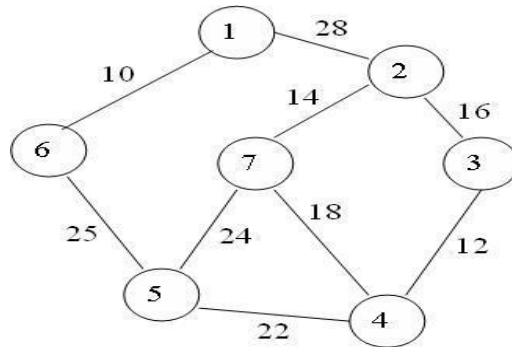
- Analysis of electrical circuit.
- Shortest route problems.

Minimum cost spanning tree:

- The cost of a spanning tree is the sum of cost of the edges in that trees.
- There are 2 method to determine a minimum cost spanning tree are
- Kruskal s Algorithm
- Prom s Algorithm.

▪ PRIM'S ALGORITHM

Start from an arbitrary vertex (root). At each stage, add a new branch (edge) to the tree already constructed; the algorithm halts when all the vertices in the graph have



Algorithm prims(e, cost, n, t){

Let (k,l) be an edge of minimum cost in E;

Mincost :=cost[k,l];

T[1,1]:=k; t[1,2]:=l;

for l:=1 to n do

 If (cost[i,l]<cost[i,k]) then near[i]:=l;

 else near[i]:=k;

Near[k]:=near[l]:=0;

For i:=2 to n-1 do {

Let j be an index such that near[j]≠0 and

 Cost[j,near[j]] is minimum;

 T[i,1]:=j; t[i,2]:=near[j];

 Mincost:=mincost+ Cost[j,near[j]];

```

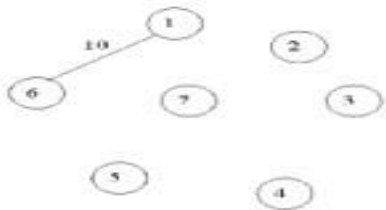
Near[j]:=0;
For k:=0 to n do
    If near((near[k]≠0) and (Cost[k,near[k]]>cost[k,j])) then
        Near[k]:=j;
}
Return mincost;
}

```

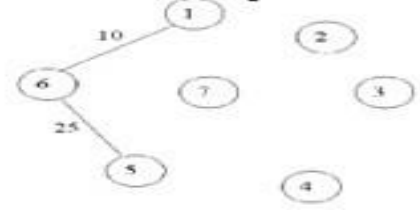
The prim's algorithm will start with a tree that includes only a minimum cost edge of G.

- ✓ Then, edges are added to the tree one by one. the next edge (i,j) to be added in such that i is a vertex included in the tree, j is a vertex not yet included, and cost of (i,j), $\text{cost}[i,j]$ is minimum among all the edges.
- ✓ The working of prim's will be explained by following diagram

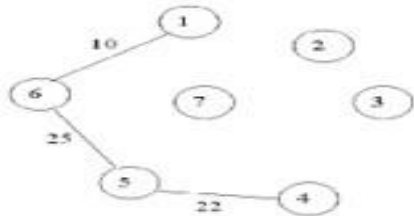
Step 1:



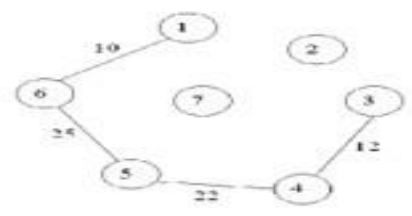
Step 2:



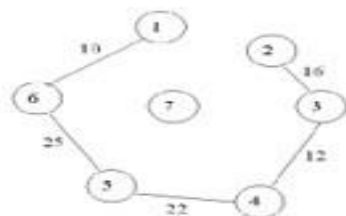
Step 3:



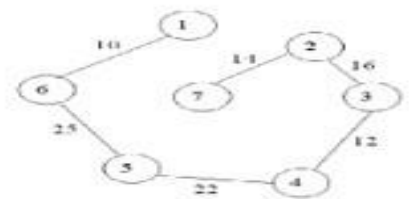
Step 4:



Step 5:



Step 6:



▪ **KRUSKAL'S ALGORITHM:**

In Kruskal's algorithm the selection function chooses edges in increasing order of length without worrying too much about their connection to previously chosen edges, except that never to form a cycle. The result is a forest of trees that grows until all the trees in a forest (all the components) merge in a single tree.

- In this algorithm, a minimum cost-spanning tree T is built edge by edge.
- Edge are considered for inclusion in T in increasing order of their cost.
 - An edge is included in T if it doesn't form a cycle with edge already in T .
 - To find the minimum cost spanning tree the edge is inserted to tree in increasing order of their cost

Algorithm:

Algorithm Kruskal(E, cost, n, t) {

// E → set of edges in G has n vertices.

// $\text{cost}[u, v]$ → cost of edge (u, v) . t → set of edge in minimum cost spanning tree

//the first cost is returned.

for $i=1$ to n do

parent[I]=-1;

$I=0$; mincost=0.0;

While(($I < n-1$) and (heap
not empty)) do

{

$j = \text{find}(u)$;

$k = \text{find}(v)$;

if(j not equal k) then {

$i=i+1$

$t[i, 1]=u$;

$t[i, 2]=v$;

mincost=mincost+cost[u, v];

union(j, k);

} }

if(i not equal $n-1$) then write(No
spanning tree) else return

minimum cost;

}

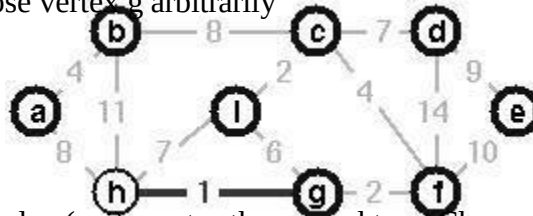
Analysis

- The time complexity of minimum cost spanning tree algorithm in worst case is $O(|E|\log|E|)$,
- where E is the edge set of G .

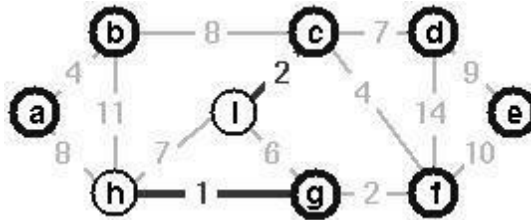
Example: Step by Step operation of Kruskal algorithm.

Step 1. In the graph, the Edge (g, h) is shortest. Either vertex g or vertex h could be representative.

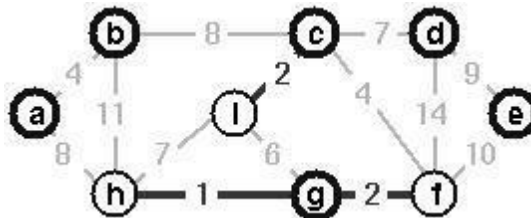
Let's choose vertex g arbitrarily



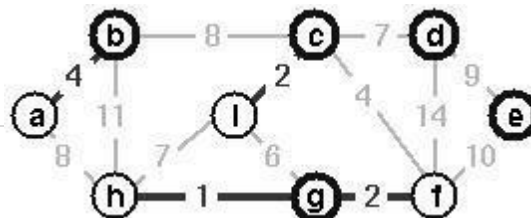
Step 2. The edge (c, i) creates the second tree. Choose vertex c as representative for second tree.

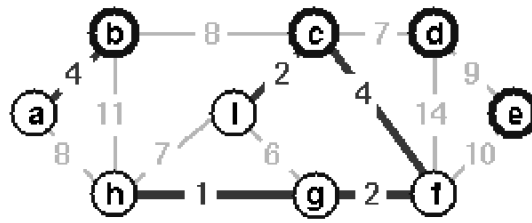


Step 3. Edge (g, g) is the next shortest edge. Add this edge and choose vertex g as representative.

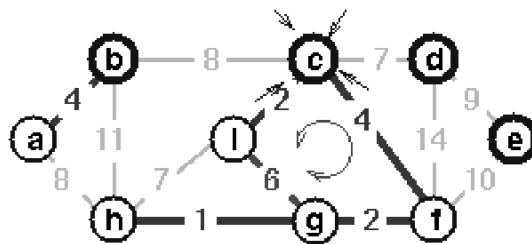


Step 4. Edge (a, b) creates a third tree.



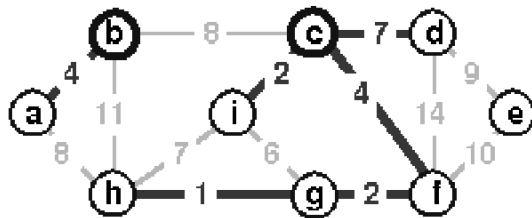


Step 5. Add edge (c, f) and merge two trees. Vertex c is chosen as the representative.

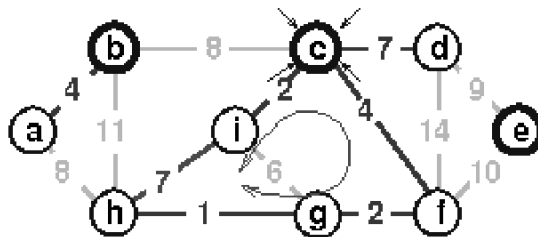


Step 6. Edge (g, i) is the next next cheapest, but if we add this edge a cycle would be created.

Vertex c is the representative of both.

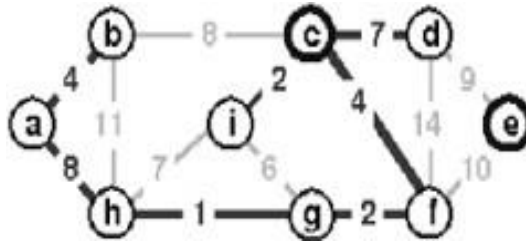


Step 7. Instead, add edge (c, d).

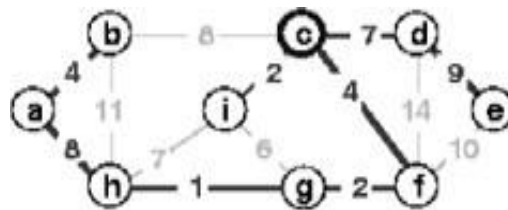


Step 8. If we add edge (h, i), edge(h, i) would make a cycle.

Step 9. Instead of adding edge (h, i) add edge (a, h).



Step 10. Again, if we add edge (b, c), it would create a cycle. Add edge (d, e) instead to complete the spanning tree. In this spanning tree all trees joined and vertex c is a sole representative.



○ Optimal Merge Patterns

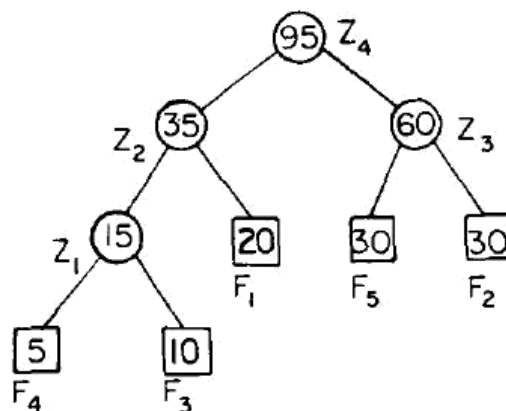
Optimal Merge Patterns

In merging two sorted files containing n and m records respectively could be merged together to obtain one sorted file in time $O(n + m)$. When more than two sorted files are to be merged together the merge can be accomplished by repeatedly merging sorted files in pairs. Thus, if files X_1 , X_2 , X_3 and X_4 are to be merged we could first merge X_1 and X_2 to get a file Y_1 . Then we could merge Y_1 and X_3 to get Y_2 . Finally, Y_2 and X_4 could be merged to obtain the desired sorted file. Alternatively, we could first merge X_1 and X_2 getting Y_1 , then merge X_3 and X_4 getting Y_2 and finally Y_1 and Y_2 getting the desired sorted file. Given n sorted files there are many ways in which to pair wise merge them into a single sorted file. Different pairings require differing amounts of computing time. Here determining an optimal (i.e. one requiring the fewest comparisons) way to pair wise merge n sorted files together.

Example: X_1 , X_2 and X_3 are three sorted files of length 30, 20 and 10 records each. Merging X_1 and X_2 requires 50 record moves. Merging the result with X_3 requires another 60 moves. The total number of record moves required to merge the three files this

way is 110. If instead, we first merge X2 and X3 (taking 30 moves) and then X1 (taking 60 moves), the total record moves made is only 90. Hence, the second merge pattern is faster than the first.

A greedy attempt to obtain an optimal merge pattern is easy to formulate. Since merging an n record file and an m record file requires possibly $n + m$ records moves, the obvious choice for a selection criterion is: at each step merge the two smallest size files together. Thus, if we have five files $F_1, \dots, F_5$ with sizes (20, 30, 10, 5, 30) our greedy rule would generate the following merge pattern: merge F_4 and F_3 to get Z_1 ($|Z_1| = 15$); merge Z_1 and F_1 to get Z_2 ($|Z_2| = 35$); merge F_5 and F_2 to get Z_3 ($|Z_3| = 60$); merge Z_2 and Z_3 to get the answer Z_4 . The total number of record moves is 205. One can verify that this is an optimal merge pattern for the given problem instance.



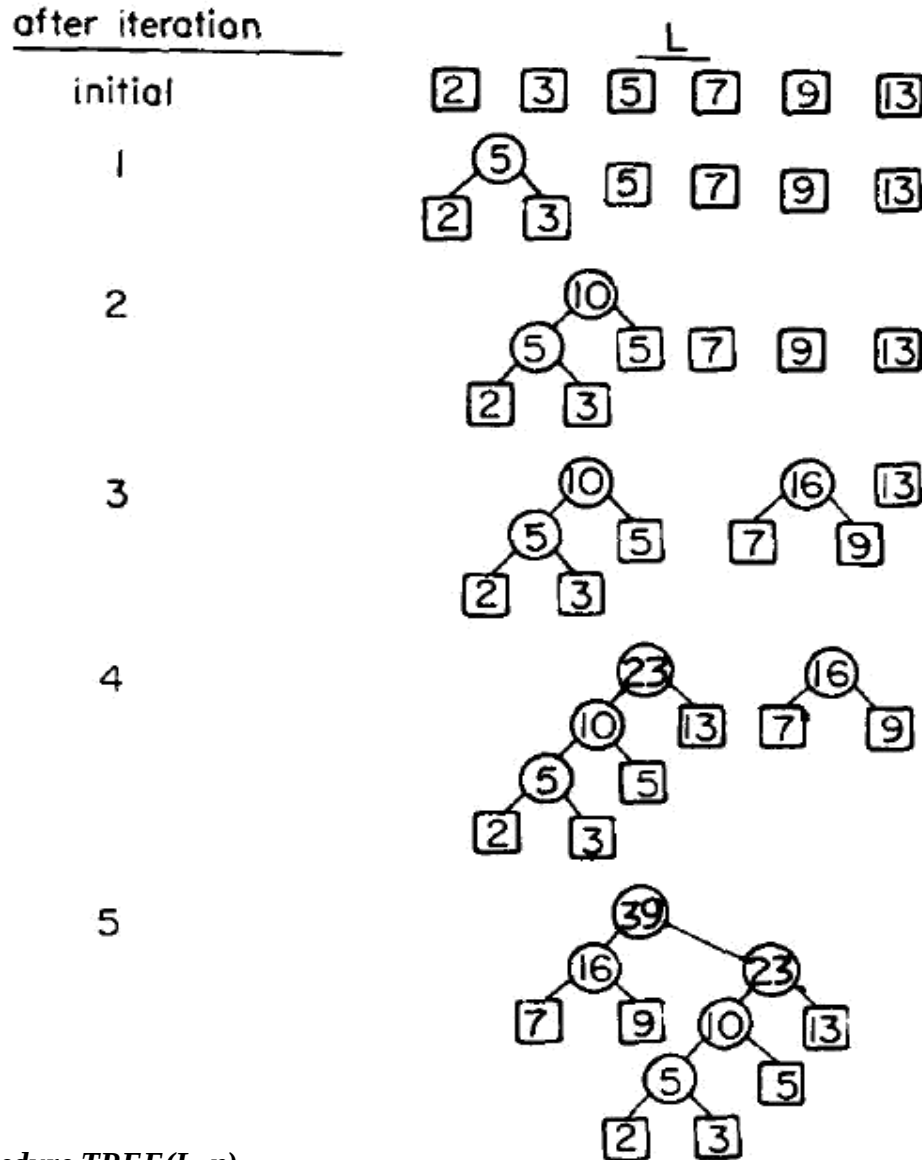
The merge pattern such as the one just described will be referred to as a 2-way merge pattern (each merge step involves the merging of two files). 2-way merge patterns may be represented by binary merge trees. Figure shows a binary merge tree representing the optimal merge pattern obtained for the above five files. The leaf nodes are drawn as squares and represent the five files. These nodes will be called external nodes. The remaining nodes are drawn circular and are called internal nodes. Each internal node has exactly two children and it represents the file obtained by merging the files represented by its two children. The number in each node is the length (i.e., the number of records) of the file represented by that node.

The external node F_4 is at a distance of 3 from the root node Z_4 (a node at level i is at a distance of $i - 1$ from the root). Hence, the records of file F_4 will be moved

three times, once to get Z1, once again to get Z2 and finally one more time to get Z4 • If d_i is the distance from the root to the external node for file F; and q_i the length of F; then the total number of record moves for this binary merge tree is

$$\sum_{i=1}^n d_i q_i.$$

This sum is called the weighted external path length of the tree.



line procedure *TREE*(*L*, *n*)

//*L* is a list of *n* single node binary trees as described above//

1 for *i* - 1 to *n* - 1 do

2 call *GETNODE*(*T*) //merge two trees with//

```

3 LCHILD(T) - LEAST(L) //smallest lengths//
4 RCHILD(T) - LEAST(L)
5 WEIGHT(T)- WEIGHT(LCHILD(T)) + WEIGHT(RCHILD(T))
6 call INSER T(L, T)
7 repeat
8 return (LEAST(L)) //tree left in L is the merge tree//
9 end TREE

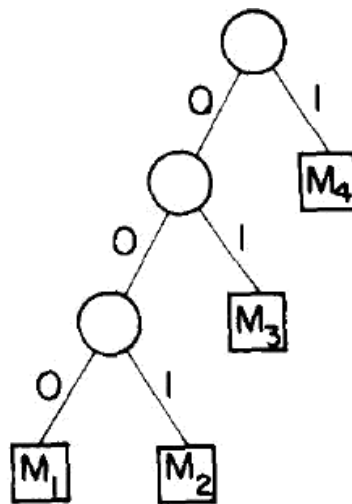
```

.Huffman Tree

Another application of binary trees with minimal weighted external path length is to obtain an optimal set of codes for messages $M_1, \dots, M_{n+1}$. Each code is a binary string which will be

used for transmission of the corresponding message. At the receiving end the code will be decoded using a decode tree. A decode tree is a binary tree in which external nodes represent messages. The binary bits in the code word for a message determine the branching needed at each level of the decode tree to reach the correct external node. For example, if we interpret a zero as a left branch and a one as a right branch, then the decode tree of Figure corresponds to codes 000, 001, 01, and 1 for messages M_1 , M_2 , M_3 and M_4 respectively. These codes are called Huffman codes.

The cost of decoding a code word is proportional to the number of bits in the code. This number is equal to the distance of the corresponding external node from the root node. If q_i is the relative frequency with which message M_i will be transmitted, then the expected decode time is $\sum_{i=1}^{n+1} q_i d_i$, where d_i is the distance of the external node for message M_i from the root node. The expected decode time is minimized by choosing code words resulting in a decode tree with minimal weighted **external path length! Note that $\sum_{i=1}^{n+1} q_i d_i$** is also the expected length of a transmitted message. Hence the code which minimizes expected decode time also minimizes the expected length of a message.

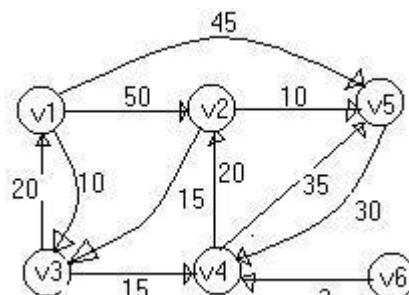


- **Single Source Shortest Path Algorithm (Dijkstra's)**

Single-source shortest path:

Graphs can be used to represent the highway structure of a state or country with vertices representing cities and edges representing sections of highway. The edges can then be assigned weights which may be either the distance between the two cities connected by the edge or the average time to drive along that section of highway. A motorist wishing to drive from city A to B would be interested in answers to the following questions:

- ✓ Is there a path from A to B?
- ✓ If there is more than one path from A to B? Which is the shortest path?



The problems defined by these questions are special case of the path problem we study in this section. The length of a path is now defined to be the sum of the weights of the edges on that path. The starting vertex of the path is referred to as the source and the last vertex the destination. The graphs are digraphs representing streets. Consider a digraph $G=(V,E)$,

with the distance to be traveled as weights on the edges. The problem is to determine the shortest path from v_0 to all the remaining vertices of G . It is assumed that all the weights associated with the edges are positive. The shortest path between v_0 and some other node v is an ordering among a subset of the edges. Hence this problem fits the ordering paradigm.

Example:

Consider the digraph of above figure. Let the numbers on the edges be the costs of travelling along that route. If a person is interested travel from v_1 to v_2 , then he encounters many paths. Some of them are:

$$v_1 - v_2 = 50 \text{ units}$$

$$v_1 - v_3 - v_4 - v_2 = 10 + 15 + 20 = 45 \text{ units}$$

$$v_1 - v_5 - v_4 - v_2 = 45 + 30 + 20 = 95 \text{ units}$$

$$v_1 - v_3 - v_4 - v_5 - v_4 - v_2 = 10 + 15 + 35 + 30 + 20 = 110 \text{ units}$$

UNIT – Four

Introduction to Dynamic programming

DYNAMIC PROGRAMMING

Dynamic programming, like the divide-and-conquer method, solves problems by combining the solutions to subproblems. As we saw in Chapter 2 of binary search, divide-and-conquer algorithms partition the problem into independent subproblems, solve the subproblems recursively, and then combine their solutions to solve the original problem. In contrast, dynamic programming is applicable when the subproblems are not independent, that is, when subproblems share sub subproblems. In this context, a divide-and-conquer algorithm does more work than necessary, repeatedly solving the common sub subproblems. A dynamic-programming algorithm solves every subproblem just once and then saves its answer in a table, thereby avoiding the work of recomputing the answer every time the subproblem is encountered. Dynamic programming is typically applied to optimization problems. In such problems there can be many possible solutions. Each solution has a value, and we wish to find a solution with the optimal (minimum or maximum) value.

Dynamic programming is a stage-wise search method suitable for optimization problems whose solutions may be viewed as the result of a sequence of decisions. The most attractive property of this strategy is that during the search for a solution it avoids full enumeration by pruning early partial decision solutions that cannot possibly lead to optimal solution. In many practical situations, this strategy hits the optimal solution in a polynomial number of decision steps. However, in the worst case, such a strategy may end up performing full enumeration.

Dynamic programming takes advantage of the duplication and arrange to solve each subproblem only once, saving the solution (in table or something) for later use. The underlying idea of dynamic programming is: avoid calculating the same stuff twice, usually by keeping a table of known results of subproblems. Unlike divide-and-conquer, which solves the subproblems top-down, a dynamic programming is a bottom-up technique.

Bottom-up means

- i. Start with the smallest sub problems.
- ii. Combining their solutions obtain the solutions to sub problems of increasing size.
- iii. Until arrive at the solution of the original problem.

The Principle of Optimality

The dynamic programming relies on a principle of optimality. This principle states that in an optimal sequence of decisions or choices, each subsequence must also be optimal. For example, in matrix chain multiplication problem, not only the value we are interested in is optimal but all the other entries in the table are also represent optimal.

The principle can be related as follows: the optimal solution to a problem is a combination of optimal solutions to some of its sub problems.

The difficulty in turning the principle of optimally into an algorithm is that it is not usually obvious which sub problems are relevant to the problem under consideration.

General method-multistage graphs-all pair shortest path algorithm-Optimal Binary Search trees-0/1 knapsack Problem-Reliability design-traveling salesman problem

BASIC SEARCH AND TRAVERSAL TECHNIQUES

The techniques-and/or graphs-biconnected components-depth first search- -breadth first search.

DYNAMIC PROGRAMING

- The idea of dynamic programming is thus quite simple: avoid calculating the same thing twice, usually by keeping a table of known result that fills up a sub instances are solved.
- Divide and conquer is a top-down method.
- When a problem is solved by divide and conquer, we immediately attack the complete instance, which we then divide into smaller and smaller sub-instances as the algorithm progresses.
- Dynamic programming on the other hand is a bottom-up technique.
- We usually start with the smallest and hence the simplest sub- instances.
- By combining their solutions, we obtain the answers to sub-instances of increasing size, until finally we arrive at the solution of the original instances.
- The essential difference between the greedy method and dynamic programming is that the greedy method only one decision sequence is ever generated.
- In dynamic programming, many decision sequences may be generated. However, sequences containing sub-optimal sub-sequences cannot be optimal and so will not be generated.

4.1. MULTISTAGE GRAPH

1. A multistage graph $G = (V, E)$ is a directed graph in which the vertices are portioned into $K \geq 2$ disjoint sets V_i , $1 \leq i \leq k$.
2. In addition, if $\langle u, v \rangle$ is an edge in E , then $u \in V_i$ and $v \in V_{i+1}$ for some i , $1 \leq i < k$.
3. If there will be only one vertex, then the sets V_i and V_k are such that $[V_i] = [V_k] = 1$.
4. Let 's' and 't' be the source and destination respectively.
5. The cost of a path from source (s) to destination (t) is the sum of the costs of the edges on the path.
6. The *MULTISTAGE GRAPH* problem is to find a minimum cost path from 's' to 't'.
7. Each set V_i defines a stage in the graph. Every path from 's' to 't' starts in stage-1, goes to stage-2 then to stage-3, then to stage-4, and so on, and terminates in stage-k.
8. This *MULTISTAGE GRAPH* problem can be solved in 2 ways.

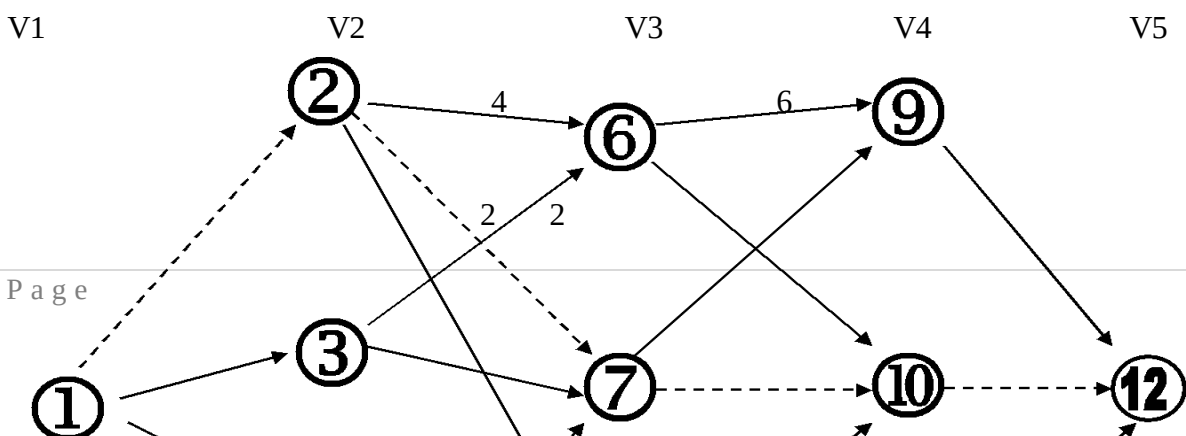
a) Forward Method.

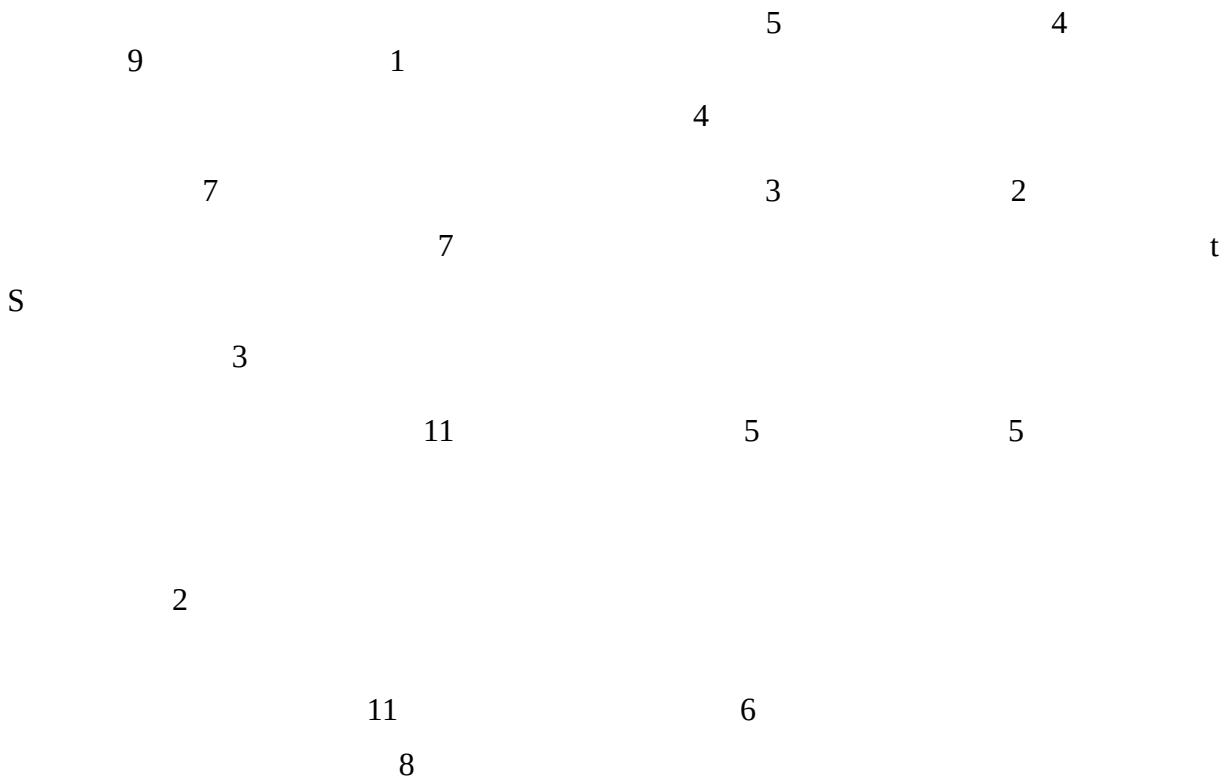
b) Backward Method.

FORWARD METHOD

1. Assume that there are 'k' stages in a graph.
2. In this *FORWARD* approach, we will find out the cost of each and every node starting from the 'k'th stage to the 1st stage.
3. We will find out the path (i.e.) minimum cost path from source to the destination (i.e.) [Stage-1 to Stage-k].

PROCEDURE:





- ❖ Maintain a cost matrix cost (n) which stores the distance from any vertex to the destination.
- ❖ If a vertex is having more than one path, then we have to choose the minimum distance path and the intermediate vertex, which gives the minimum distance path, will be stored in the distance array 'D'.
- ❖ In this way we will find out the minimum cost path from each and every vertex.
- ❖ Finally cost (1) will give the shortest distance from source to destination.
- ❖ For finding the path, start from vertex-1 then the distance array D(1) will give the minimum cost neighbor vertex which in turn give the next nearest vertex and proceed in this way till we reach the Destination.
- ❖ For a 'k' stage graph, there will be 'k' vertex in the path.
- ❖ In the above graph V1...V5 represent the stages. This 5-stage graph can be solved by using forward approach as follows,

STEPS: -

Cost (12)=0

Cost (11)=5

Cost (10)=2

Cost (9)=4

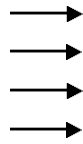
DESTINATION, D

D (12)=0

D (11)=12

D (10)=12

D (9)=12



❖ For forward approach,

$$\text{Cost (i,j) = min \{C (j,l) + Cost (i+1,l) \}}$$

$$l \in V_{i+1}$$

$$(j,l) \in E$$

$$\text{Cost(8) = min \{C (8,10) + Cost (10), C (8,11) + Cost (11) \}}$$

$$= \text{min (5 + 2, 6 + 5)}$$

$$= \text{min (7,11)}$$

$$= 7$$

$$\text{cost(8) = 7} \Rightarrow \text{D(8)=10}$$

$$\text{cost(7) = min(c (7,9)+ cost(9),c (7,10)+ cost(10))}$$

$$= \text{min (4+4,3+2)}$$

$$= \text{min(8,5)}$$

$$= 5$$

$$\text{cost(7) = 5} \Rightarrow \text{D(7) = 10}$$

$$\text{cost(6) = min (c (6,9) + cost(9),c (6,10) +cost(10))}$$

$$= \text{min(6+4 , 5 +2)}$$

$$= \text{min(10,7)}$$

$$\begin{aligned} &= 7 \\ \text{cost}(6) &= 7 \Rightarrow D(6) = 10 \end{aligned}$$

$$\begin{aligned}\text{cost}(5) &= \min (c(5,7) + \text{cost}(7), c(5,8) + \text{cost}(8)) \\ &= \min(11+5, 8+7) \\ &= \min(16, 15)\end{aligned}$$

$$= 15$$

$$\text{cost}(5) = 15 \Rightarrow D(5) = 18$$

$$\text{cost}(4) = \min (c(4,8) + \text{cost}(8))$$

$$= \min(11+7)=18$$

$$\text{cost}(4) = 18 \Rightarrow D(4) = 8$$

$$\begin{aligned}\text{cost}(3) &= \min (c(3,6) + \text{cost}(6), c(3,7) + \text{cost}(7)) \\ &= \min(2+7, 7+5)\end{aligned}$$

$$= \min(9, 12)=9$$

$$\text{cost}(3) = 9 \Rightarrow D(3) = 6$$

$$\begin{aligned}\text{cost}(2) &= \min (c(2,6) + \text{cost}(6), c(2,7) + \text{cost}(7), c(2,8) + \text{cost}(8)) \\ &= \min(4+7, 2+5, 1+7)\end{aligned}$$

$$= \min(11, 7, 8)=7$$

$$\text{cost}(2) = 7 \Rightarrow D(2) = 7$$

$$\begin{aligned}\text{cost}(1) &= \min (c(1,2) + \text{cost}(2), c(1,3) + \text{cost}(3), c(1,4) + \text{cost}(4), c(1,5) + \text{cost}(5)) \\ &= \min(9+7, 7+9, 3+18, 2+15)\end{aligned}$$

$$= \min(16, 16, 21, 17)$$

$$= 16$$

$$\text{cost}(1) = 16 \Rightarrow D(1) = 2$$

.....→ The path through which you have to find the shortest distance.



Start from vertex - 2

$$D(1) = 2$$

$$D(2) = 7$$

$$D(7) = 10$$

$$D(10) = 12$$

So, the minimum –cost path is, The cost is $9+2+3+2+=16$

ALGORITHM: FORWARD METHOD

Algorithm FGraph (G,k,n,p){

// The i/p is a k-stage graph $G=(V,E)$ with ‘n’ vertex.

// Indexed in order of stages E is a set of edges.

// and $c[i,j]$ is the cost of $\langle i,j \rangle$, $p[1:k]$ is a minimum cost path.

cost[n]=0.0;

for $j=n-1$ to 1 step-1 do{

//compute cost[j],

// let ‘r’ be the vertex such that $\langle j,r \rangle$ is an edge of ‘G’ &

// $c[j,r]+cost[r]$ is minimum.

cost[j] = $c[j+r] + cost[r]$;

d[j] =r;

} // find a minimum cost path.

P[1]=1;

P[k]=n;

For $j=2$ to $k-1$ do

P[j]=d[p[j-1]];

}

ANALYSIS:

The time complexity of this forward method is $O(|V| + |E|)$ **BACKWARD METHOD**

- ❖ If there are 'K' stages in a graph using back ward approach. we will find out the cost of each & every vertex starting from 1st stage to the kth stage.
- ❖ We will find out the minimum cost path from destination to source (i.e.,) from stage k to stage 1.

PROCEDURE:

1. It is similar to forward approach, but differs only in two or three ways.
2. Maintain a cost matrix to store the cost of every vertices and a distance matrix to store the minimum distance vertex.
3. Find out the cost of each and every vertex starting from vertex 1 up to vertex k.
4. To find out the path star from vertex 'k', then the distance array D (k) will give the minimum cost neighbor vertex which in turn gives the next nearest neighbor vertex and proceed till we reach the destination.

STEP:

$$\text{Cost}(1) = 0 \Rightarrow D(1)=0$$

$$\text{Cost}(2) = 9 \Rightarrow D(2)=1$$

$$\text{Cost}(3) = 7 \Rightarrow D(3)=1$$

$$\text{Cost}(4) = 3 \Rightarrow D(4)=1$$

$$\text{Cost}(5) = 2 \Rightarrow D(5)=1$$

$$\text{Cost}(6) = \min(c(2,6) + \text{cost}(2), c(3,6) + \text{cost}(3)) = \min(13, 9)$$

$$\text{cost}(6) = 9 \Rightarrow D(6)=3$$

$$\text{Cost}(7) = \min(c(3,7) + \text{cost}(3), c(5,7) + \text{cost}(5), c(2,7) + \text{cost}(2))$$

$$= \min(14, 13, 11)$$

$$\text{cost}(7) = 11 \Rightarrow D(7)=2$$

$$\text{Cost}(8) = \min(c(2,8) + \text{cost}(2), c(4,8) + \text{cost}(4), c(5,8) + \text{cost}(5))$$

$$= \min(10, 14, 10)$$

$$\text{cost}(8) = 10 \Rightarrow D(8)=2$$

$$\text{Cost}(9) = \min(c(6,9) + \text{cost}(6), c(7,9) + \text{cost}(7))$$

$$= \min(15, 15)$$

$$\text{cost}(9) = 15 \Rightarrow D(9) = 6$$

$$\text{Cost}(10) = \min(c(6,10) + \text{cost}(6), c(7,10) + \text{cost}(7), c(8,10) + \text{cost}(8)) = \min(14, 14, 15)$$

$$\text{cost}(10) = 14 \Rightarrow D(10) = 6$$

$$\text{Cost}(11) = \min(c(8,11) + \text{cost}(8))$$

$$\text{cost}(11) = 16 \Rightarrow D(11) = 8$$

$$\text{cost}(12) = \min(c(9,12) + \text{cost}(9), c(10,12) + \text{cost}(10), c(11,12) + \text{cost}(11))$$

$$= \min(19, 16, 21)$$

$$\text{cost}(12) = 16 \Rightarrow D(12) = 10$$

PATH:

Start from vertex-12

$$D(12) = 10$$

$$D(10) = 6$$

$$D(6) = 3$$

$$D(3) = 1$$

So the minimum cost is, $= 7 + 2 + 5 + 2 = 16$. The cost is 16.

ALGORITHM : BACKWARD METHOD

Algorithm BGraph (G,k,n,p) {

// The I/p is a k-stage graph $G=(V,E)$ with 'n' vertex.

// Indexed in order of stages E is a set of edges.

// and $c[i,j]$ is the cost of $\langle i,j \rangle$, $p[1:k]$ is a minimum cost path.

bcost[1]=0.0;

for j=2 to n do

{

```

//compute bcost[j],
//    let 'r' be the vertex such that <r,j> is an edge of 'G' &

//    bcost[r]+c[r,j] is minimum.

bcost[j] = bcost[r] + c[r,j];
d[j] =r;

}
// find a minimum cost path.
P[1]=1;

P[k]=n;
For j= k-1 to 2 do

P[j]=d[p[j+1]];
}

```

4.2. ALL PAIR SHORTEST PATH

- Let $G=\langle V,E \rangle$ be a directed graph 'V' is a set of nodes and 'E' is the set of edges.
- Each edge has an associated non-negative length.
- We want to calculate the length of the shortest path between each pair of nodes.
- Suppose the nodes of G are numbered from 1 to n, so $V=\{1,2,...N\}$, and suppose G matrix L gives the length of each edge, with $L(i,j)=0$ for $i=1,2,...n, L(i,j) \geq 0$ for all i & j, and $L(i,j)=\text{infinity}$, if the edge (i,j) does not exist.
- The principle of optimality applies: if k is the node on the shortest path from i to j then the part of the path from i to k and the part from k to j must also be optimal, that is shorter.
- First, create a cost adjacency matrix for the given graph.
- Copy the above matrix-to-matrix D, which will give the direct distance between nodes.
- We have to perform N iteration after iteration k. the matrix D will give you the

distance between nodes with only $(1, 2, \dots, k)$ as intermediate nodes.

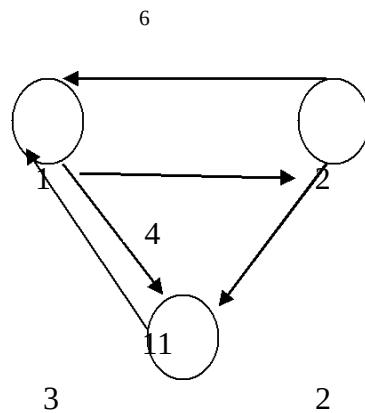
- At the iteration k, we have to check for each pair of nodes (i,j) whether or not there exists a path from i to j passing through node k.

$$A^k(i,j) = \min\{ A^{k-1}(i,j), A^{k-1}(i,k) + A^{k-1}(k,j) \} \quad k \geq 1$$

Let $G=(V,E)$ is a directed graph with 'N' Vertices. Let cost be the cost of the adjacency matrix for G such that $\text{cost}(i,i)=0$ ($1 \leq i \leq n$) then $\text{cost}(i,j)$ is the length or cost of the edge $\langle i,j \rangle$.

If $\langle i,j \rangle$ exists the edge belongs to $E(G)$ and $\text{cost}(i,j)=\alpha$, if $i \neq j$ and $\langle i,j \rangle \notin E(G)$

Example



Matrix Representation:

$$A^0 = \begin{bmatrix} 0 & 4 & 11 \\ 6 & 0 & 2 \\ 3 & \alpha & 0 \end{bmatrix}$$

The path of the vertices through vertex-1 is given as

$$A^1 = \begin{bmatrix} 0 & 4 & 11 \\ 6 & 0 & 2 \\ 3 & 7 & 0 \end{bmatrix}$$

The path of the vertices through vertex-2 is given as

$$A^2 = \begin{matrix} & 0 & 4 & \mathbf{6} \\ 6 & 6 & 0 & 2 \\ 3 & 3 & 7 & 0 \end{matrix}$$

The path of the vertices through vertex-3 is given as

$$A^3 = \begin{matrix} & 0 & 4 & 11 \\ \mathbf{5} & 5 & 0 & 2 \\ 3 & 3 & 7 & 0 \end{matrix}$$

At 1st iteration we have to check the each pair(i,j) whether there is a path through node 1. if so we have to check whether it is minimum than the previous value and if it is so then the distance through 1 is the value of d1(i,j). at the same time we have to solve the intermediate node in the matrix position p(i,j).

ALGORITHM:

Function AllPair (L[1..r,1..r]):array[1..n,1..n]

array D[1..n,1..n]

D = L

For k = 1 to n do

For i = 1 to n do

For j = 1 to n do

D[i, j] = min (D[i, j], D[i, k] + D[k, j])

Return D

ANALYSIS:

This algorithm takes a time of $O(n^3)$

4.3. OPTIMAL BINARY SEARCH TREES

If n-identifiers exists, every internal node represents a P and every external node represents a point where an unsuccessful search may terminate.

The cost of searching an identifier ai is $P(i) * \text{level}(a_i)$

The cost of unsuccessful search which terminates in E_i is $q(i) * (\text{level}(E_i) - 1)$

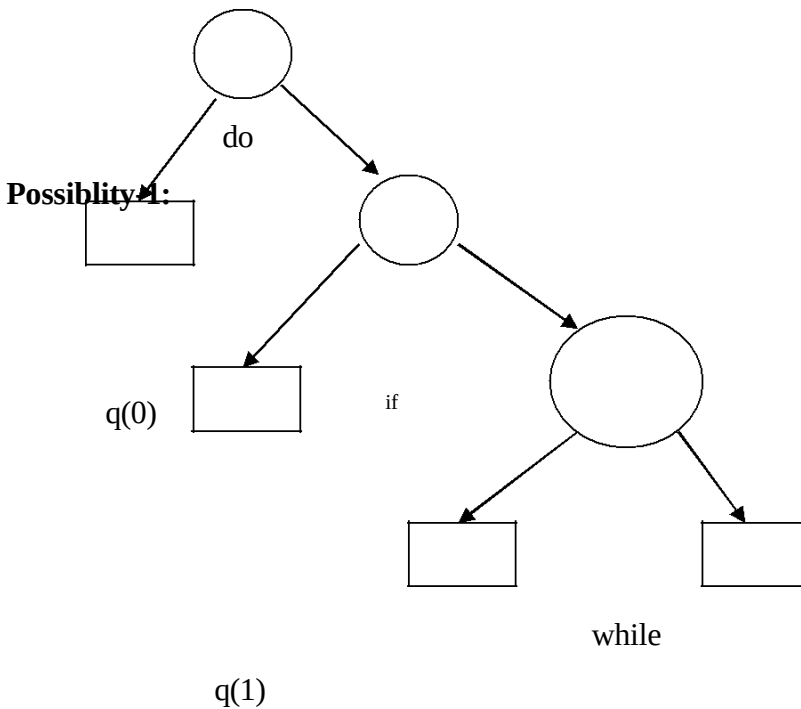
Therefore, the cost of the binary search tree is

$$\sum_{1 \leq i \leq n} (i) * level(ai) + \sum_{0 \leq i \leq n} q(i) * (level(Ei) - 1)$$

Criterion for an optimal tree (principle of optimality): Each optimal binary search tree is composed of a root and (at most) two optimal subtrees, the left and the right. Or, stated in another way: in an optimal tree, all the subtrees must also be optimal with respect to the keys that they contain.

Example: Find OBST for a set $\{a_1, a_2, a_3\} = \{\text{do}, \text{if}, \text{while}\}$ if $P(1)=0.5, P(2)=0.1, P(3)=0.05$ and $q(0)=0.15, q(1)=0.1, q(2)=0.05, q(3)=0.05$

Solution: The possible binary trees are many.

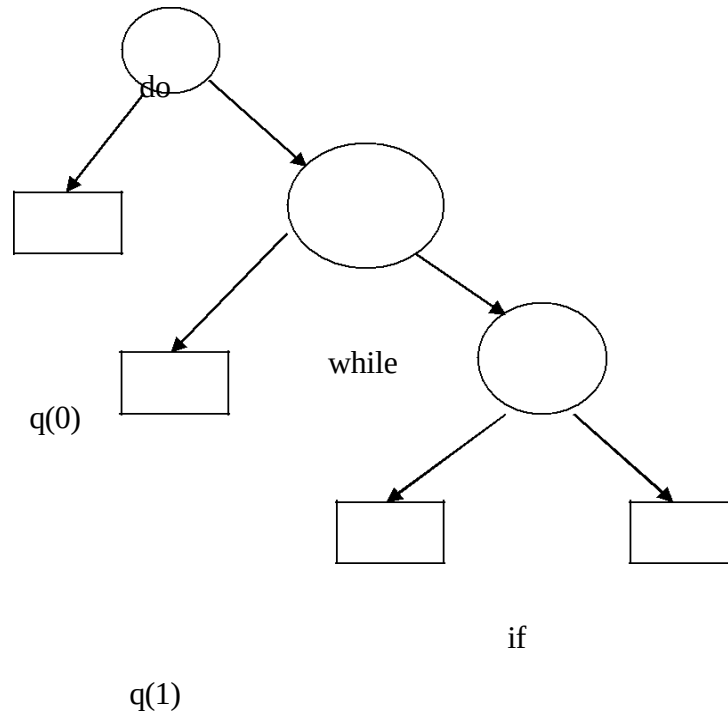


q(2)

q(3)

$$\text{Cost}=(0.5*1+0.1*2+0.05*3)+(0.15*1+0.1*2+0.05*3+0.05*3)=1.50$$

Possiblity-2:



q(2)

q(3)

$$\text{Cost}=(0.5*1+0.05*2+0.1*3)+(0.15*1+0.1*2+0.05*3+0.05*3)=0.9+0.65=1.7$$

Possiblity-3:

Possiblity-4:

:

:

We get many possible binary search trees and out of which first tree is having the minimum cost i.e., the optimal binary search tree.

Example1: Compute $w(i,j)$, $c(i,j)$, $r(i,j)$ for the identifier set $n = 4$, $(a_1, a_2, a_3, a_4) = \{\text{do}, \text{if}, \text{int}, \text{while}\}$ with $P(1:4) = \{3, 3, 1, 1\}$ and $q = \{2, 3, 1, 1, 1\}$. Using the $r(i,j)$ s construct the optimal binary search

tree.

Solution:

$$w(i, j) = P(j) + q(j) + w(i, j - 1)$$

$$\text{cost}(i, j) = \min \{ \text{cost}(i, k-1) + \text{cost}(k, j) \} + w(i, j)$$

$$i < k \leq j$$

$$w(i, i) = q(i), C(i, i) = r(i, i) = 0$$

The 'K' value which gives minimum cost is chosen as 'r'.

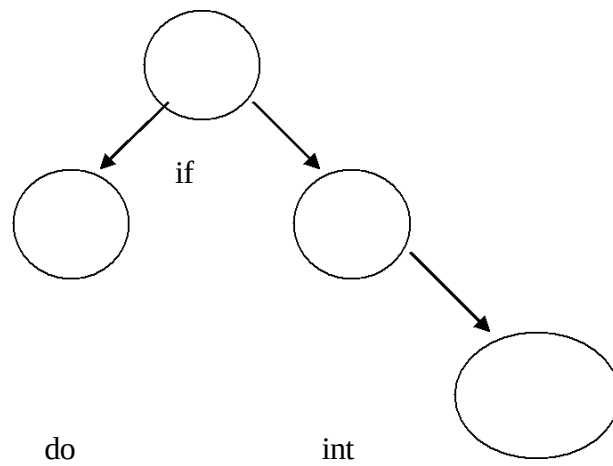
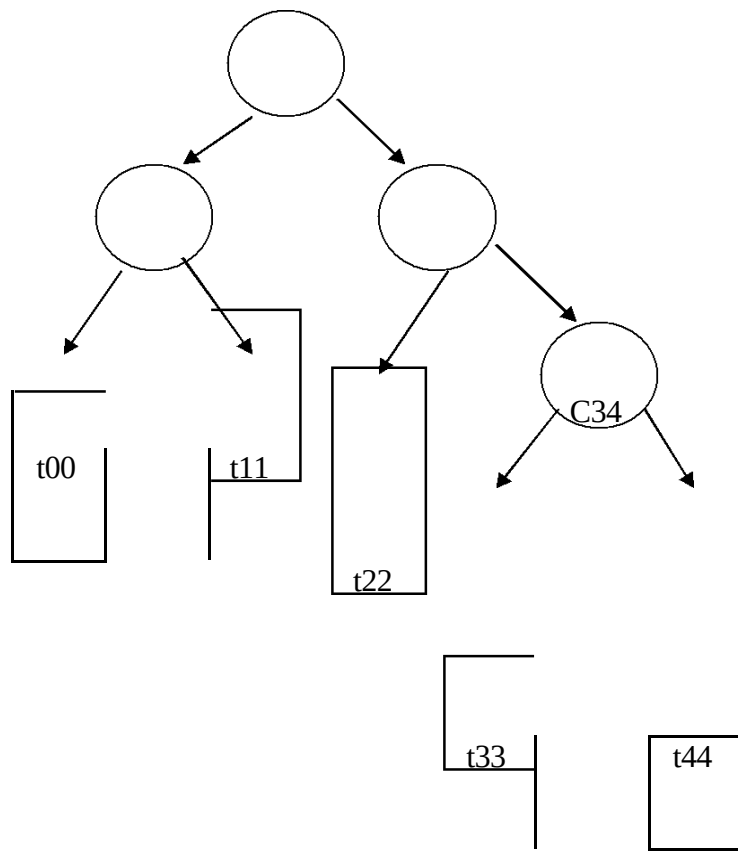
	0	1	2	3	4
j-i=0	W(0,0)=2 C(0,0)=0 r(0,0)=0	W(1,1)=3 C(1,1)=0 r(1,1)=0	W(2,2)=1 C(2,2)=0 r(2,2)=0	W(3,3)=1 C(3,3)=0 r(3,3)=0	W(4,4)=1 C(4,4)=0 r(4,4)=0
j-i=1	W(0,1)=8 C(0,1)=8 r(0,1)=1	W(1,2)=7 C(1,2)=7 r(1,2)=2	W(2,3)=3 C(2,3)=3 r(2,3)=3	W(3,4)=3 C(3,4)=3 r(3,4)=4	
j-i=2	W(0,2)=12 C(0,2)=19 r(0,2)=1	W(1,3)=9 C(1,3)=12 r(1,3)=2	W(2,4)=5 C(2,4)=8 r(2,4)=3		
j-i=3	W(0,3)=14 C(0,3)=25 r(0,3)=2	W(1,4)=11 C(1,4)=19 r(1,4)=2			
j-i=4	W(0,4)=16 C(0,4)=32 r(0,4)=2				

This algorithm takes a time of $O(n^3)$.

C04

C01

C24



while

4.4. 0/1 KNAPSACK PROBLEM

- This problem is similar to ordinary knapsack problem but we may not take a fraction of an object.
- We are given 'N' objects with weight W_i and profits P_i where i varies from 1 to N and also a knapsack with capacity 'M'.
- The problem is, we have to fill the bag with the help of 'N' objects and the resulting profit has to be maximum.
- Formally, the problem can be stated as, maximize $\sum_{i=1}^n X_i P_i$

$$\text{subject to } \sum_{i=1}^n X_i W_i \leq M$$

- Where X_i are constraints on the solution $X_i \in \{0,1\}$. X_i is required to be 0 or 1. If the object is selected then the unit is 1. If the object is rejected then the unit is 0. That is why it is called as 0/1, knapsack problem.
- Time complexity is $O(nW)$. n is the number of items and W is the weight of the knapsack.
- For X_n , two states are possible

- o If X_n is included in the knapsack then the remaining capacity is $M - W_n$ and profit earned is P_n .
- o If X_n is not included in the knapsack then capacity remains M and no profit is earned.

Therefore, we have to select a state such that it maximizes the result. This can be shown as

$$f_n(M) = \text{Max} \{ f_{n-1}(x), f_{n-1}(M - W_n) + P_n \}$$

When $f_n(M)$ —maximizing profit function for object ‘n’ where capacity is M.

This can be generalized as

$$f_i(y) = \text{Max} \{ f_i(y), f_{i-1}(y - W_i) + P_i \}$$

We solve 0/1 knapsack problem using the above equation.

By generating set of ordered pairs (P, W) where $P = f_i(y)$,

$W = y_i$ The set S^{i+1} is computed from S^i , where

$$S_1^i = \{ (P, W) | (P - P_i, (W - W_i)) \in S^i \}$$

S^{i+1} is obtained by merging and purging the order pairs of S^i and S_1^i . Purging is done by applying dominance rule.

Dominance Rule: If S^{i+1} contains two ordered pairs (P_j, W_j) , (P_k, W_k) such that $P_j < P_k$ and $W_j > W_k$ then the ordered pair (P_j, W_j) is discarded.

Example: Given $n=3$ and $M=6$, $(P_1, P_2, P_3) = (1, 2, 5)$, $(W_1, W_2, W_3) = (2, 3, 4)$. Find the optimal solution for 0/1 knapsack.

Solution:

$$S^0 = \{(0, 0)\}$$

$$S_1^0 = \{ (1 + 0, 2 + 0) \}$$

$$= \{(1, 2)\}$$

$$S^1 = \{(0, 0), (1, 2)\}$$

Apply dominance rule for purging

$$S_1^1 = \{ (2 + 0, 3 + 0), (2 + 1, 3 + 2) \}$$

$$= \{(2, 3), (3, 5)\}$$

$$S^2 = \{(0, 0), (1, 2), (2, 3), (3, 5)\}$$

Apply dominance rule for purging

$$S_1^2 = \{(5 + 0, 4 + 0), (5 + 1, 4 + 2), (5 + 2, 4 + 3), (5 + 3, 4 + 5)\}$$

$$=\{(5,4),(6,6),(7,7),(8,9)\}$$

$$S^3 = \{(0,0),(1,2),(2,3),(3,5), (5,4),(6,6),(7,7),(8,9)\}$$

(7,7) , (8,9) are discarded, as maximum capacity is only 6.

Apply dominance rule for purging. (3,5) is discarded using purging ($3 < 5, 5 > 4$) as we need maximum profit.

$$\text{Hence, } S^3 = \{(0,0),(1,2),(2,3), (5,4),(6,6)\}$$

Select the largest ordered pair i.e., (6,6)

$$X_3 = (6,6) \in S^3$$

$$(6,6) \notin S^2$$

Therefore, $X_3 = 1$

This implies the knapsack is filled now and the profit is 6. For next ordered pair, remove 3rd object profit and weight.

$$\text{i.e., } (6-5, 6-4) = (1,2)$$

The next ordered pair is (1,2) as adding (5,4) is giving (6,6)

$$X_2 = (1,2) \in S^2$$

$$(1,2) \in S^1$$

Therefore, $X_2 = 0$. It cannot be placed.

The next ordered pair is (1,2)

$$X_1 = (1,2) \in S^1$$

$$(1,2) \notin S^0$$

Therefore, $X_1 = 1$.

The optimal solution is $(x_1, x_2, x_3) = (1, 0, 1)$

4.5. RELIABILITY DESIGN

The problem is to design a system that is composed of several devices connected in series. Let r_i be the reliability of device D_i (that is r_i is the probability that device i will function properly) then the reliability of the entire system is $\prod r_i$. Even if the individual devices are very reliable (the r_i 's are very close to one), the reliability of the system may not be very good. For example, if $n = 10$ and $r_i = 0.99$, $1 \leq i \leq 10$, then $\prod r_i = .904$.

Hence, it is desirable to duplicate devices. Multiple copies of the same device type are connected in parallel. If stage i contains m_i copies of device D_i . Then the probability that all m_i has a malfunction is $(1 - r_i)^{m_i}$. Hence the reliability of stage i becomes $1 - (1 - r_i)^{m_i}$. The reliability of stage ' i ' is given by a function $\Phi_i(m_i)$.

Our problem is to use device duplication. This maximization is to be carried out under a cost constraint. Let c_i be the cost of each unit of device i and let c be the maximum allowable cost of the system being designed.

We wish to solve:

Maximize

$$\prod_{1 \leq i \leq n} \Phi_i(m_i)$$

Subject to

$$\sum_{1 \leq i \leq n} C_i m_i \leq C$$

$$m_i \geq 1$$

$$m_i > 1 \text{ and interger, } 1 < i < n$$

Example 1:

Design a three-stage system with device types D_1 , D_2 and D_3 . The costs are 30 Birr, 15 Birr and 20 Birr respectively. The Cost of the system is to be no more than 105 Birr. The reliability of each device is 0.9, 0.8 and 0.5 respectively.

Solution : The reliability design is the problem of setting up of system with a given cost, such that the reliability of system is maximize

- Example given **Total cost is 105**

D_i	C_i	R_i	U_i
D_1	30	0.9	2
D_2	15	0.8	3
D_3	20	0.5	3

To find upper bound of each device

$$U_i = \left(\frac{C - \sum C_i}{C_i} \right) + 1$$

$$\sum C_i = C_1 + C_2 + C_3 = 30 + 15 + 20 = 65$$

$$U_1 = \frac{105 - 65}{30} + 1 = 2$$

$$U_2 =$$

30

$$\frac{105}{65} -$$

$$S^1 = (\text{Merge } S_1^1, S_1^1)$$

$$S^2 = (\text{Merge } S^2, S^2, S^2)$$

$$\frac{105-65}{3^{15}} + 1 =$$

$$U3 = \frac{\quad}{20} + 1 = 3$$

NB: at least one copy of each device must be there so that the system can setup.

The remaining cost is 40, using the remaining cost 40 how many copies of each device we shall take such that reliability of system maximum.

It follows **set Method: Set of order pair (R,C), initially reliability is 1, and cost is 0,(1,0)**

4.6. TRAVELLING SALESMAN PROBLEM

- Let $G(V,E)$ be a directed graph with edge cost c_{ij} is defined such that $c_{ij} > 0$ for all i and j and $C_{ij} = \infty$, if $\langle i,j \rangle \notin E$.

Let $|V|=n$ and assume $n > 1$.

- The traveling salesman problem is to find a tour of minimum cost.
- A tour of G is a directed cycle that include every vertex in V .
- The cost of the tour is the sum of cost of the edges on the tour.
- The tour is the shortest path that starts and ends at the same vertex i.e., 1.

APPLICATION:

1. Suppose we have to route a postal van to pick up mail from the mail boxes located at 'n' different sites.
2. An $n+1$ vertex graph can be used to represent the situation.
3. One vertex represents the post office from which the postal van starts and returns.
4. Edge $\langle i,j \rangle$ is assigned a cost equal to the distance from site 'i' to site 'j'.
5. The route taken by the postal van is a tour and we are finding a tour of minimum length.

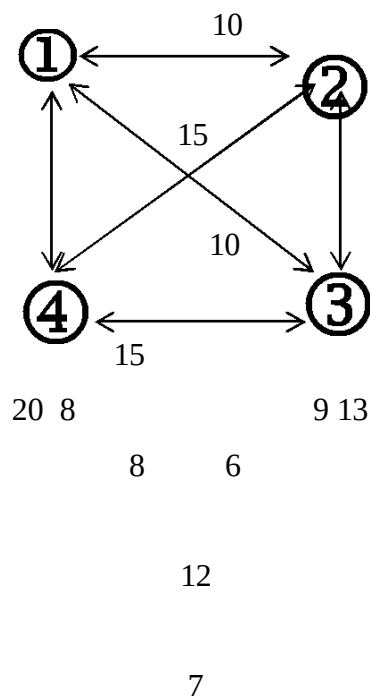
6. Every tour consists of an edge $\langle 1, k \rangle$ for some $k \in V - \{1\}$ and a path from vertex k to vertex 1.
7. The path from vertex k to vertex 1 goes through each vertex in $V - \{1, k\}$ exactly once.
8. The function which is used to find the path is

$$g(1, V - \{1\}) = \min \{ c_{ij} + g(j, S - \{j\}) \}$$
9. $g(i, S)$ be the length of a shortest path starting at vertex i , going through all vertices in S and terminating at vertex 1.
10. The function $g(1, V - \{1\})$ is the length of an optimal tour.

STEPS TO FIND THE PATH:

1. Find $g(i, \Phi) = c_{i1}$, $1 \leq i \leq n$, hence we can use equation (2) to obtain $g(i, S)$ for all S to size 1.
2. That we have to start with $s=1$, (ie) there will be only one vertex in set ' s '.
3. Then $s=2$, and we have to proceed until $|s| \leq n-1$.

Example1:



Cost matrix

0	10	15	20
5	0	9	10

6 13 0 12

8 8 9 0

$g(i,s)$ $\longrightarrow$ set of nodes/vertex have to visited.
 $\downarrow$

starting position

$$g(i,s) = \min\{c_{ij} + g(j,s-\{j\})\}$$

STEP 1:

$$\begin{aligned} g(1,\{2,3,4\}) &= \min\{c_{12} + g(2,\{3,4\}), c_{13} + g(3,\{2,4\}), c_{14} + g(4,\{2,3\})\} \\ &= \min\{10+25, 15+25, 20+23\} \\ &= \min\{35, 35, 43\} \\ &= 35 \end{aligned}$$

STEP 2:

$$\begin{aligned} g(2,\{3,4\}) &= \min\{c_{23} + g(3,\{4\}), c_{24} + g(4,\{3\})\} \\ &= \min\{9+20, 10+15\} \\ &= \min\{29, 25\} \\ &= 25 \end{aligned}$$

$$\begin{aligned} g(3,\{2,4\}) &= \min\{c_{32} + g(2,\{4\}), c_{34} + g(4,\{2\})\} \\ &= \min\{13+18, 12+13\} \\ &= \min\{31, 25\} \\ &= 25 \end{aligned}$$

$$g(4,\{2,3\}) = \min\{c_{42} + g(2,\{3\}), c_{43} + g(3,\{2\})\}$$

$$\min\{8+15, 9+18\}$$

$$\min\{23,27\}$$

$$=23$$

STEP 3:

$$\begin{aligned} 1. \quad g(3, \{4\}) &= \min\{c_{34} + g(4, \Phi)\} \\ &= 12 + 8 = 20 \end{aligned}$$

$$\begin{aligned} 2. \quad g(4, \{3\}) &= \min\{c_{43} + g(3, \Phi)\} \\ &= 9 + 6 = 15 \end{aligned}$$

$$\begin{aligned} 3. \quad g(2, \{4\}) &= \min\{c_{24} + g(4, \Phi)\} \\ &= 10 + 8 = 18 \end{aligned}$$

$$\begin{aligned} 4. \quad g(4, \{2\}) &= \min\{c_{42} + g(2, \Phi)\} \\ &= 8 + 5 = 13 \end{aligned}$$

$$\begin{aligned} 5. \quad g(2, \{3\}) &= \min\{c_{23} + g(3, \Phi)\} \\ &= 9 + 6 = 15 \end{aligned}$$

$$\begin{aligned} 6. \quad g(3, \{2\}) &= \min\{c_{32} + g(2, \Phi)\} \\ &= 13 + 5 = 18 \end{aligned}$$

STEP 4:

$$g(4, \Phi) = c_{41} = 8$$

$$g(3, \Phi) = c_{31} = 6$$

$$g(2, \Phi) = c_{21} = 5$$

$$|S| = 0$$

i = 1 to n.

$$g(1, \Phi) = c_{11} \Rightarrow 0$$

$$g(2, \Phi) = c_{21} \Rightarrow 5$$

$$g(3, \Phi) = c_{31} \Rightarrow 6$$

$$g(4, \Phi) = c_{41} \Rightarrow 8$$

$$|S|=1$$

$$i = 2 \text{ to } 4$$

$$g(2, \{3\}) = c_{23} + g(3, \Phi)$$

$$= 9+6 = 15$$

$$g(2, \{4\}) = c_{24} + g(4, \Phi)$$

$$= 10+8 = 18$$

$$g(3, \{2\}) = c_{32} + g(2, \Phi)$$

$$= 13+5 = 18$$

$$g(3, \{4\}) = c_{34} + g(4, \Phi)$$

$$= 12+8 = 20$$

$$g(4, \{2\}) = c_{42} + g(2, \Phi)$$

$$= 8+5 = 13$$

$$g(4, \{3\}) = c_{43} + g(3, \Phi)$$

$$= 9+6 = 15$$

$$|S| = 2$$

$$i \neq 1, 1 \in s \text{ and } i \in s.$$

$$g(2, \{3,4\}) = \min\{c_{23}+g(3\{4\}), c_{24}+g(4, \{3\})\}$$

$$\min\{9+20, 10+15\}$$

$$\min\{29, 25\}$$

$$= 25$$

$$g(3, \{2,4\}) = \min\{c_{32}+g(2\{4\}), c_{34}+g(4, \{2\})\}$$

$$\min\{13+18, 12+13\}$$

$$\min\{31, 25\}$$

$$= 25$$

$$g(4, \{2,3\}) = \min\{c_{42}+g(2\{3\}), c_{43}+g(3, \{2\})\}$$

$$\min\{8+15, 9+18\}$$

$$\min\{23, 27\}$$

$$= 23$$

$$|S| = 3$$

$$g(1,\{2,3,4\})=\min\{c_{12}+g(2\{3,4\}),c_{13}+g(3,\{2,4\}),$$

$$c_{14}+g(4,\{2,3\})\} \min\{10+25,15+25,20+23\}$$

$$\min\{35,35,43\}$$

$$=35$$

optimal cost is 35

the shortest path is,

$$g(1,\{2,3,4\}) = c_{12} + g(2,\{3,4\}) \Rightarrow 1 \rightarrow 2$$

$$g(2,\{3,4\}) = c_{24} + g(4,\{3\}) \Rightarrow 1 \rightarrow 2 \rightarrow 4$$

$$g(4,\{3\}) = c_{43} + g(3\{\}) \Rightarrow 1 \rightarrow 2 \rightarrow 4 \rightarrow 3 \rightarrow 1$$

so the optimal tour is **1→2→4→3→1**

4.7. BASIC SEARCH AND TRAVERSAL TECHNIQUE

DEFINING GRAPH:

A graph G consists of a set V of vertices (nodes) and a set E of edges (arcs). We write $G=(V,E)$. V is a finite and non-empty set of vertices. E is a set of pair of vertices; these pairs are called as edges. Therefore, $V(G)$ is read as V of G , is a set of vertices and $E(G)$, read as E of G is a set of edges.

An edge $e=(v, w)$ is a pair of vertices v and w , and to be incident with v and w .

A graph can be pictorially represented as follows,

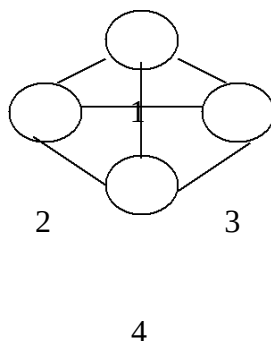


FIG: Graph G

We have numbered the graph as 1,2,3,4. Therefore, $V(G)=(1,2,3,4)$ and $E(G) = \{(1,2),(1,3),(1,4),(2,3),(2,4)\}$.

BASIC TERMINOLOGIES OF GRAPH:

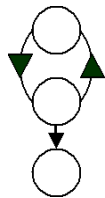
UNDIRECTED GRAPH:

An undirected graph is that in which, the pair of vertices representing the edges is unordered.

DIRECTED GRAPH:

An directed graph is that in which, each edge is an ordered pair of vertices, (i.e.) each edge is represented by a directed pair. It is also referred to as digraph.

DIRECTED GRAPH



COMPLETE GRAPH:

An n vertex undirected graph with exactly $n(n-1)/2$ edges is said to be complete graph. The graph G is said to be complete graph.

TECHNIQUES FOR GRAPHS:

- The fundamental problem concerning graphs is the reach-ability problem.
- In it simplest form it requires us to determine whether there exist a path in the given graph, $G=(V,E)$ such that this path starts at vertex 'v' and ends at vertex 'u'.

- A more general form is to determine for a given starting vertex $v \in V$ all vertex 'u' such that there is a path from v to u .
- This problem can be solved by starting at vertex 'v' and systematically searching the graph 'G' for vertex that can be reached from 'v'.
- We describe 2 search methods for this.
 - i. Breadth first Search and Traversal.
 - ii. Depth first Search and Traversal.

BREADTH FIRST SEARCH AND TRAVERSAL

Breadth First Search:

In Breadth first search we start at vertex v and mark it as having been reached. The vertex v at this time is said to be unexplored. A vertex is said to have been explored by an algorithm when the algorithm has visited all vertices adjacent from it. All unvisited vertices adjacent from v are visited next. There are new unexplored vertices. Vertex v has now been explored. The newly visited vertices have not been explored and are put onto the end of the list of unexplored vertices. The first vertex on this list is the next to be explored. Exploration continues until no unexplored vertex is left. The list of unexplored vertices acts as a queue and can be represented using any of the standard queue representations.

- In Breadth First Search we start at a vertex 'v' and mark it as having been reached (visited).
- The vertex 'v' is at this time said to be unexplored.
- A vertex is said to have been explored by an algorithm when the algorithm has visited all vertices adjacent from it.
- All unvisited vertices adjacent from 'v' are visited next. These are new unexplored vertices.
- Vertex 'v' has now been explored. The newly visit vertices have not been explored and are put on the end of a list of unexplored vertices.
- The first vertex on this list in the next to be explored. Exploration continues until no unexplored vertex is left.

- The list of unexplored vertices operates as a queue and can be represented using any of the start queue representation.

ALGORITHM:

Algorithm BFS (v)

```
// A breadth first search of 'G' is carried out.
// beginning at vertex-v; For any node i, visit.

// if 'i' has already been visited. The graph 'v'
// and array visited [] are global; visited []

// initialized to zero.

{ y=v; // q is a queue of unexplored 1visited (v)=1
repeat

{ for all vertices 'w' adjacent from u do

    { if (visited[w]=0) then
    {Add w to q;

    visited[w]=1
    }

    }

if q is empty then return;// No delete u from q;

} until (false)
}
```

algorithim : breadth first traversal

algorithm BFS(G, n)

```

{

    for i= 1 to n do
        visited[i] =0;

    for i =1 to n do
        if (visited[i]=0)then BFS(i)

}

```

Here the time and space required by BFT on an n-vertex e-edge graph one $O(n+e)$ and $O(n)$ resp. if adjacency list and adjacency matrix is used then the bounds are $O(n^2)$ and $O(n)$ resp.

DEPTH FIRST SEARCH

A depth first search of a graph differs from a breadth first search in that the exploration of a vertex v is suspended as soon as a new vertex is reached. At this time the exploration of the new vertex u begins. When this new vertex has been explored, the exploration of u continues. The search terminates when all reached vertices have been fully explored. This search process is best-described recursively.

```

Algorithm DFS(v){

    visited[v]=1
    for each vertex w adjacent from v do{
        If (visited[w]=0)then

            DFS(w);
        }

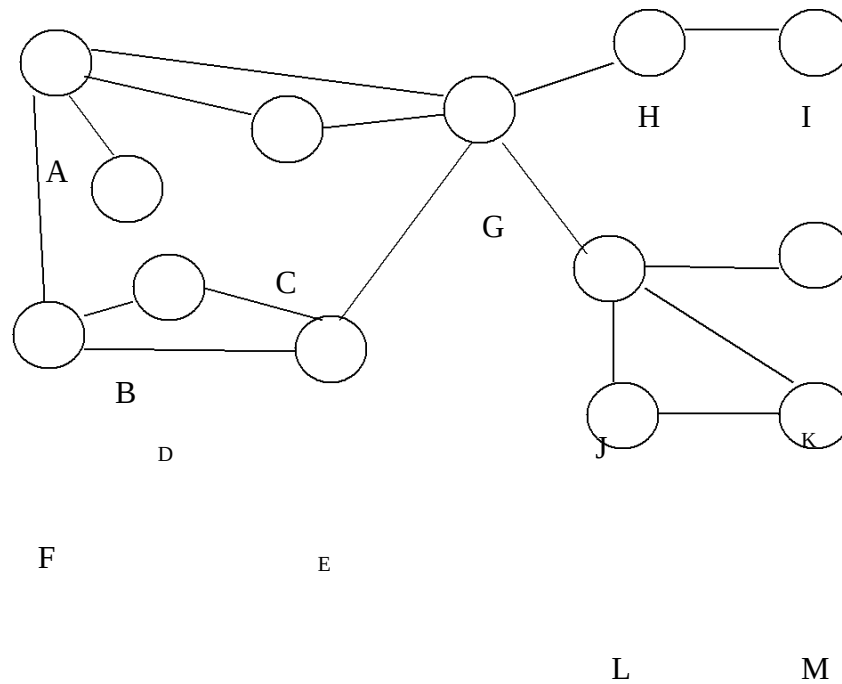
    }
}

```

BI-CONNECTED COMPONENT AND DFS

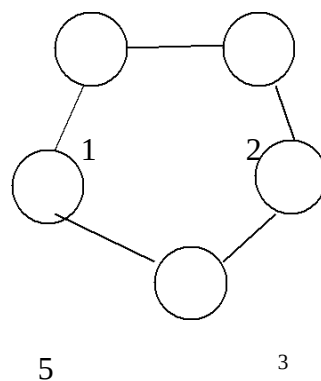
Articulation Point: An articulation point in a connected graph is a vertex that if deleted, would break the graph into two or more pieces (Connected components)

Considering the below graph



A,G,H,J are the articulation points for the above graph.

Bi-connected graph: A graph with no articulation point is said to be bi-connected graph. A graph is bi-connected iff any vertex is deleted; the graph remains connected.



Bi-connected Component: A bi-connected component of a graph is a maximal bi-connected subgraph. A graph which is not bi-connected can divide into bi-connected components set of nodes mutually accessible via two distinct paths.

From the above graph {A,G,E,F},{J,L,M},{K,I,B} are bi-connected components.

How to find articulation point:

1. Find the depth first spanning tree T for graph G.
2. Add back edges represented as $--\square$ in T. Back edges are the edges which have no connection in DFS but there is an edge existing in original graph.
3. Finding Articulation point Nodes
 - a. If 'i' is the root of T and has atleast 2 children, then 'i' is an articulation point.
 - b. Leaf nodes are not articulation points
 - c. 'i' is an articulation point, if 'i' is not the root and has a child which does not connect to the existing vertices even with back edges.

UNIT – Five

5. Introduction to Backtracking

5.1 Backtracking:

- It is one of the most general algorithm design techniques.
- Many problems which deal with searching for a set of solutions or for a optimal solution satisfying some constraints can be solved using the backtracking formulation.
- To apply backtracking method, the desired solution must be expressible as an n-tuple $(x_1 \dots x_n)$ where x_i is chosen from some finite set S_i .
- The problem is to find a vector, which maximizes or minimizes a criterion function $P(x_1 \dots x_n)$.
- The major advantage of this method is, once we know that a partial vector $(x_1, \dots, x_i)$ will not lead to an optimal solution that $(m_{i+1} \dots m_n)$ possible test vectors may be ignored entirely.
- Many problems solved using backtracking require that all the solutions satisfy a complex set of constraints.
- These constraints are classified as:
 - 1) **Explicit constraints.**
 - 2) **Implicit constraints.**

1)Explicit constraints:

Explicit constraints are rules that restrict each X_i to take values only from a given set.

Some examples are,

$X_i \geq 0$ or $S_i = \{\text{all non-negative real nos.}\}$

$X_i = 0 \text{ or } 1$ or $S_i = \{0, 1\}$.

$L_i \leq X_i \leq U_i$ or $S_i = \{a: L_i \leq a \leq U_i\}$

- All tuples that satisfy the explicit constraint define a possible solution space for 'i'.

2)Implicit constraints:

The implicit constraint determines which of the tuples in the solution space 'i' can actually satisfy the criterion functions.

Algorithm:

Algorithm Backtracking (n)

// This schema describes the backtracking procedure. All solutions are generated in $X[1:n]$

//and printed as soon as they are determined.

{

k=1;

While (k $\neq$ 0) do {

if (there remains all untried

$X[k] \in T(X[1], [2], \dots, X[k-1])$ and $B_k(X[1], \dots, X[k])$ is true) then

{

if($X[1], \dots, X[k]$) is the path to the answer node)

Then write($X[1:k]$);

k=k+1; //consider the next step.

}

else k=k-1; //consider backtracking to the previous set.

}}

- All solutions are generated in $X[1:n]$ and printed as soon as they are determined.
- $T(X[1], \dots, X[k-1])$ is all possible values of $X[k]$ gives that $X[1], \dots, X[k-1]$ have already been chosen.
- $B_k(X[1], \dots, X[k])$ is a **boundary function** which determines the elements of
- $X[k]$ which satisfies the implicit constraint.
- Certain problems which are solved using backtracking method are
 1. **N-Queens problem.**
 2. **Graph coloring.**

3. Hamiltonian cycle.

5.2 8-Queens Problem:

This 8 queens' problem is to place n-queens in an 'N*N' matrix in such a way that no two queens attack each other otherwise no two queens should be in the same row, column, diagonal.

Solution:

- ❖ The solution vector $X (X_1 \dots X_n)$ represents a solution in which X_i is the column of the i^{th} row where i^{th} queen is placed.
- ❖ First, we have to check no two queens are in same row.
- ❖ Second, we have to check no two queens are in same column.
- ❖ The function, which is used to check these two conditions, is $[I, X(j)]$, which gives position of the i^{th} queen, where I represents the row and $X(j)$ represents the column position.
- ❖ Third, we have to check no two queens are in it diagonal.
- ❖ Consider two-dimensional array $A [1: n, 1: n]$ in which we observe that every element on the same diagonal that runs from upper left to lower right has the same value.
- ❖ Also, every element on the same diagonal that runs from lower right to upper left has the same value.
- ❖ Suppose two queens are in same position (i,j) and (k,l) then two queens lie on the same diagonal, if and only if $|j-l|=|I-k|$.

STEPS TO GENERATE THE SOLUTION:

- ❖ Initialize x array to zero and start by placing the first queen in $k=1$ in the first row.
- ❖ To find the column position start from value 1 to n , where ' n ' is the no. Of columns or no. Of queens.
- ❖ If $k=1$ then $x(k)=1$. so $(k, x(k))$ will give the position of the k^{th} queen. Here we have to check whether there is any queen in the same column or diagonal.
- ❖ For this considers the previous position, which had already, been found out. Check whether $X(I)=X(k)$ for column $|X(i)-X(k)|=(I-k)$ for the same diagonal.
- ❖ If any one of the conditions is true then return false indicating that k^{th} queen can't be placed in position $X(k)$.
- ❖ For not possible condition increment $X(k)$ value by one and precede d until the position is found.
- ❖ If the position $X(k) \leq n$ and $k=n$ then the solution is generated completely.

- ❖ If $k < n$, then increment the 'k' value and find position of the next queen.

- ❖ If the position $X(k) > n$ then k^{th} queen cannot be placed as the size of the matrix is ' $N*N$ '.
- ❖ So decrement the ' k ' value by one i.e. we have to back track and after the position of the previous queen.

Algorithm:

Algorithm place (k, I)

//return true if a queen can be placed in k^{th} row and i^{th} column. otherwise it returns //
 //false. $X[]$ is a global array whose first $k-1$ values have been set. $\text{Abs}()$ returns the
 //absolute value of r .

```
{
  For  $j=1$  to  $k-1$  do

    If  $((X[j]=I) \quad //\text{two in same column.}$ 
    Or  $(\text{abs}(X[j]-I)=\text{Abs}(j-k)))$ 

  Then return false;
  Return true;

}
```

Example: 4 queens.

Q

1			

(a)

1			
.	.	2	

(b)

1			
		2	
.	.	.	.

(c)

1			
			2
.	3		

(d)

Solutin-1

(2 4 1 3)

1			
			2
	3		
.	.	.	.

(e)

	1		

(f)

	1		
.	.	.	2

(g)

	1		
			2
3			
.	.	4	

(h)

		Q	
Q			
			Q

Q

Solution 2

(3 1 4 2)

Example: One solution to 8 queens.

Two possible solutions are

	Q		
			Q

			Q				
					Q		
							Q

Q

Q

Q

Q

Q

Q

5.3 Sum of Subsets

- The Sum of Subset Problem is, there will be a set of distinct positive Numbers X and a given Number N. Now, we have to find all the combination of numbers from X that sum up to N. Make a Set of those Number.

- If we sort the weights in nondecreasing order before doing the search, there is an obvious sign telling us that a node is nonpromising.

- Bounding function

- $B_k(x_1, \dots, x_k) = \text{true}$ iff

$$\sum_{i=1}^k w_i x_i + \sum_{i=k+1}^n w_i \geq m$$

- Assuming w_i 's in nondecreasing order, $(x_1, \dots, x_k)$ cannot lead to an answer node if

$$\sum_{i=1}^k w_i x_i + w_{k+1} > m$$

- So, the bounding function is

$$B_k(x_1, \dots, x_k) = \text{true} \text{ iff } \sum_{i=1}^k w_i x_i + \sum_{i=k+1}^n w_i \geq m$$

$$\text{and } \sum_{i=1}^k w_i x_i + w_{k+1} \leq m$$

- Example: Show the pruned state space tree when backtracking is used with $n = 4$, $W = 13$, and $w_1 = 3$, $w_2 = 4$, $w_3 = 5$, and $w_4 = 6$.

5.4 Graph Coloring:

- ❖ Let 'G' be a graph and 'm' be a given positive integer. If the nodes of 'G' can be colored in such a way that no two adjacent nodes have the same color. Yet only 'M' colors are used.

- ❖ There are Two Problems:

- The **m-coloring optimization problem**, calls for coloring a graph using the minimum number of coloring.

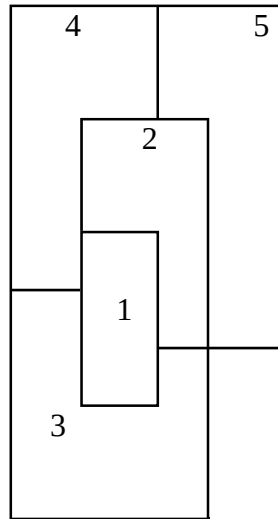
- The **m-coloring decision problem**, known as K-coloring asks whether a graph is colourable using at most k-colors.

- ❖ The graph G can be colored using the smallest integer 'm'. This integer is referred to as chromatic number of the graph.

❖ A graph is said to be planar iff it can be drawn on plane in such a way that no two edges cross each other.

❖ Suppose we are given a map then; we have to convert it into planar. Consider each and every region as a node. If two regions are adjacent then the corresponding nodes are joined by an edge.

Consider a map with five regions and its graph.



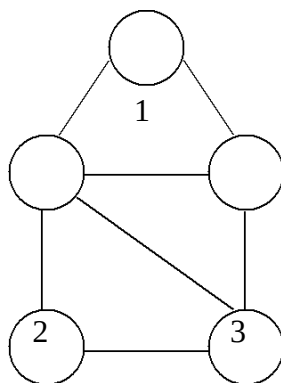
1 is adjacent to 2, 3, 4.

2 is adjacent to 1, 3, 4, 5

3 is adjacent to 1, 2, 4

4 is adjacent to 1, 2, 3, 5

5 is adjacent to 2, 4



5

4

Steps to color the Graph:

❖ First create the adjacency matrix $\text{graph}(1:m,1:n)$ for a graph, if there is an edge between i,j then $C(i,j) = 1$ otherwise $C(i,j) = 0$.

❖ The Colors will be represented by the integers $1,2,\dots,m$ and the solutions will be stored in the array $X(1),X(2),\dots,X(n)$, $X(\text{index})$ is the color, index is the node.

❖ The formula is used to set the color is,

$$X(k) = (X(k)+1) \% (m+1)$$

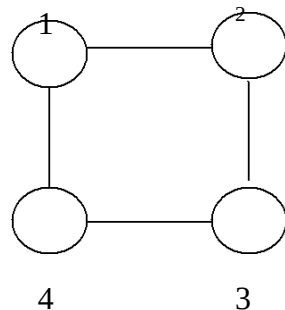
❖ First one chromatic number is assigned, after assigning a number for 'k' node, we have to check whether the adjacent nodes has got the same values if so then we have to assign the next value.

❖ Repeat the procedure until all possible combinations of colors are found.

❖ The function which is used to check the adjacent nodes and same color is,

If((Graph (k,j) == 1) and $X(k) = X(j)$)

Example:



$N = 4$

$M = 3$

Adjacency Matrix:

$$\begin{vmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \end{vmatrix}$$

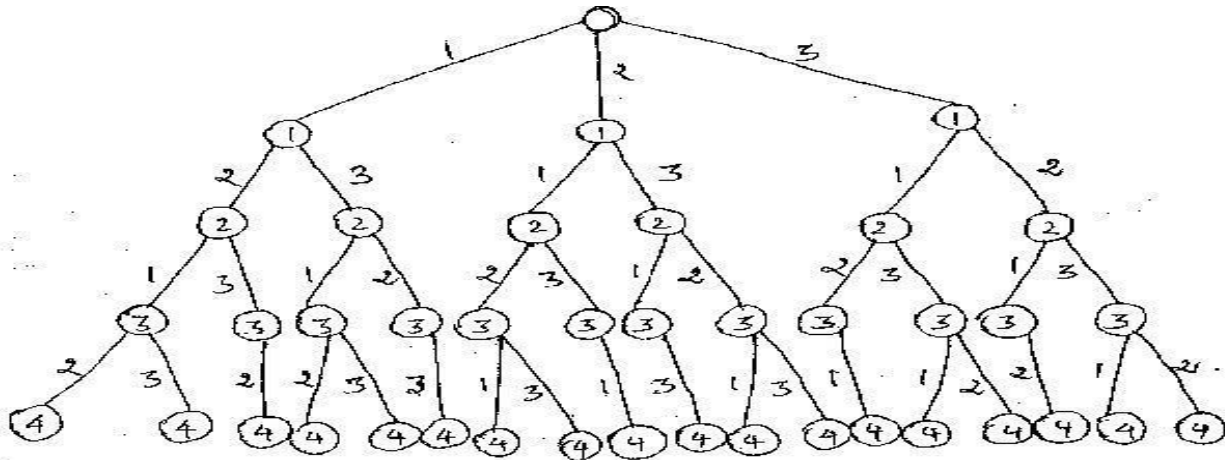
0 1 0 1

1 0 1 0

- = Problem is to color the given graph of 4 nodes using 3 colors.
- = Node-1 can take the given graph of 4 nodes using 3 colors.

$$=$$

State Space Tree:



Algorithm:

Algorithm mColoring(k)

=the graph is represented by its Boolean adjacency matrix $G[1:n,1:n]$.All assignments

//of 1,2,... ,m to the vertices of the graph such that adjacent vertices are assigned

```
//distinct integers are printed. 'k' is the index of the next vertex to color.
```

 $\{$

repeat

$$\{$$

```
// generate all legal assignment for X[k].  
Nextvalue(k); // Assign to X[k] a legal color.  
  
If (X[k]=0) then return; // No new color possible.
```

If ($k=n$) then // Almost 'm' colors have been used to color the 'n' vertices

 Write($x[1:n]$);

Else mcoloring($k+1$);

}until(false);

}

Algorithm Nextvalue(k)

// $X[1], \dots, X[k-1]$ have been assigned integer values in the range $[1, m]$ such that
// adjacent values have distinct integers. A value for $X[k]$ is determined in the
// range $[0, m]$. $X[k]$ is assigned the next highest numbers color while maintaining
// distinctness from the adjacent vertices of vertex K. If no such color exists, then $X[k]$ is 0.

{

 repeat

 {

$X[k] = (X[k]+1) \bmod (m+1)$; // next highest color.

 If ($X[k]=0$) then return; // All colors have been used.

 For $j=1$ to n do

 {

 // Check if this color is distinct from adjacent color.

 If ($(G[k,j] \neq 0) \text{ and } (X[k] = X[j])$)

 // If (k,j) is an edge and if adjacent vertices have the same
 color. Then break;

 }

 if ($j=n+1$) then return; // new color found.

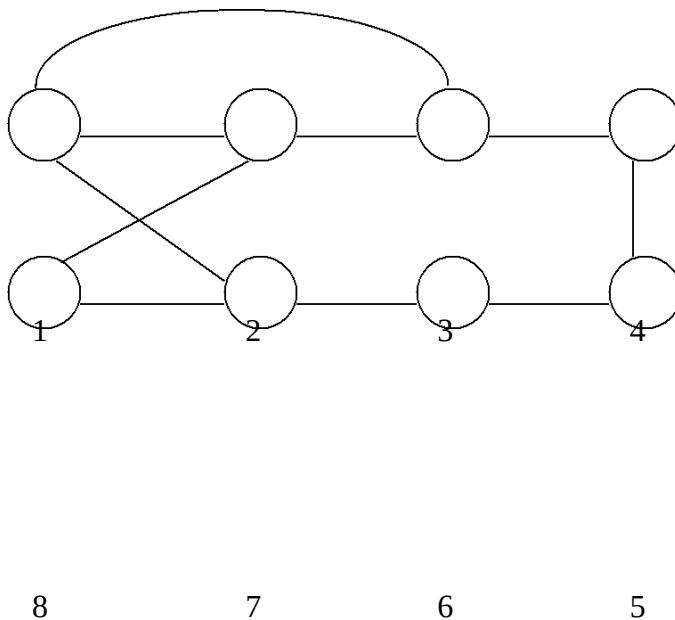
 } until(false); // otherwise try to find another color.

}

- ✓ The time spent by Next value to determine the children is $\theta(mn)$
- ✓ Total time is $=\theta(m^n * n)$.

5.5 Hamiltonian cycles:

- Let $G=(V,E)$ be a connected graph with 'n' vertices. A HAMILTONIAN CYCLE is a round trip path along 'n' edges of G which every vertex once and returns to its starting position.
- If the Hamiltonian cycle begins at some vertex V_1 belongs to G and the vertex are visited in the order of $V_1, V_2, \dots, V_{n+1}$, then the edges are in $E, 1 \leq i \leq n$ and the V_i are distinct except V_1 and V_{n+1} which are equal.
- Consider an example graph G1.



The graph G1 has Hamiltonian cycles:

->1,3,4,5,6,7,8,2,1 and

->1,2,8,7,6,5,4,3,1.

- ❖ The backtracking algorithm helps to find Hamiltonian cycle for any type of graph.
- ❖ The following graphs (figure A, B, and C) are not Hamiltonian cycle.
 - ✓ In figure A Vertex 3 connected two graphs, so, the vertex is **Articulate point** It's not Hamiltonian Cycle
 - ✓ In figure B Vertex 5, 6 are **pendant** vertex. So, It's not Hamiltonian Cycles

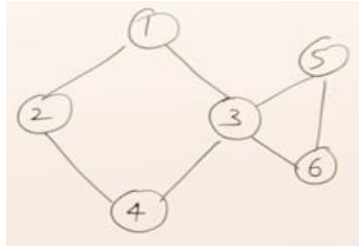


Figure (A)

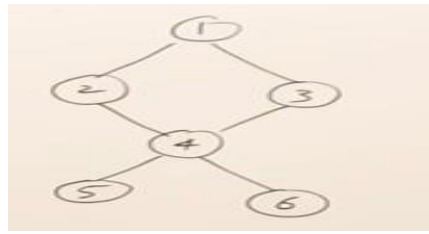


Figure (B)

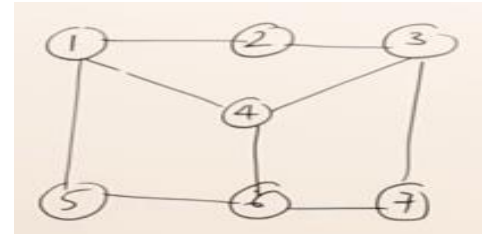


Figure (C)

Procedure:

1. Define a solution vector $X(X_1 \dots \dots X_n)$ where X_i represents the i th visited vertex of the proposed cycle.
2. Create a cost adjacency matrix for the given graph.
3. The solution array initialized to all zeros except $X(1)=1$, because the cycle should start at vertex '1'.
4. Now we have to find the second vertex to be visited in the cycle.
5. The vertex from 1 to n are included in the cycle one by one by checking 2 conditions,
 1. There should be a path from previous visited vertex to current vertex.
 2. The current vertex must be distinct and should not have been visited earlier.
6. When these two conditions are satisfied the current vertex is included in the cycle, else the next vertex is tried.
7. When the n th vertex is visited, we have to check, is there any path from n th vertex to first vertex. if no path, then go back one step and after the previous visited node.
8. Repeat the above steps to generate possible Hamiltonian cycle.

Algorithm:(Finding all Hamiltonian cycle)

Algorithm Hamiltonian (k) {

 Loop

 Next value (k)

 If ($x(k)=0$) then return;

 {

 If $k=n$ then

```
    Print (x)
else
    Hamiltonian (k+1);
```

```

End if
}
Repeat
}
Algorithm Nextvalue (k){
Repeat{
X [k]=(X [k]+1) mod (n+1); //next vertex
If (X [k]=0) then return;

If (G [X [k-1], X [k]] ≠ 0) then
{
For j=1 to k-1 do if (X [j]=X [k]) then break;

// Check for distinction.
If (j=k) then //if true then the vertex is distinct.
If ((k<n) or ((k=n) and G [X [n], X [1]] ≠ 0)) then return;
}

} Until (false);
}

```

5.6 Knapsack Problem using Backtracking

- ❖ The problem is similar to the zero-one (0/1) knapsack optimization problem is dynamic programming algorithm.
- ❖ We are given 'n' positive weights W_i and 'n' positive profits P_i , and a positive number 'm' that is the knapsack capacity, the is problem calls for choosing a subset of the weights such that,

$$\sum_{1 \leq i \leq n} W_i X_i \leq m \text{ and } // P_i X_i \text{ is Maximized.}$$

$$1 \leq i \leq n$$

$X_i \rightarrow$ Constitute Zero-one valued Vector.

- ❖ The Solution space is the same as that for the sum of subset's problem.

- ❖ Bounding functions are needed to help kill some live nodes without expanding them. A good bounding function for this problem is obtained by using an upper bound on the value of the best feasible solution obtainable by expanding the given live node.
- ❖ The profits and weights are assigned in descending order depend upon the ratio.

$$(i.e.) P_i/W_i$$

$$\geq P_{(I+1)} / W_{(I+1)}$$

Solution:

- After assigning the profit and weights, we have to take the first object weights and check if the first weight is less than or equal to the capacity, if so then we include that object (i.e.) the unit is 1.(i.e.) $K \rightarrow 1$.
- Then we are going to the next object, if the object weight is exceeded that object does not fit. So unit of that object is '0'.(i.e.) $K=0$.
- Then we are going to the bounding function, this function determines an upper bound on the best solution obtainable at level $K+1$.
- Repeat the process until we reach the optimal solution.

Algorithm:

Algorithm Bknap(k,cp,cw)

// 'm' is the size of the knapsack; 'n' $\rightarrow$ no. of weights & profits. $W[]$ & $P[]$ are the
//weights & weights. $P[I]/W[I] \geq P[I+1]/W[I+1]$.

//fw $\rightarrow$ Final weights of
knapsack.

//fp final max.profit.

//x[k] = 0 if W[k] is not the knapsack,else X[k]=1.

{ // Generate left child. If $((W+W[k] \leq m)$ then

{
Y[k] =1;

If($k < n$) then Bnap($k+1, cp+P[k], Cw$
+ $W[k]$) If($(Cp + p[w] > fp)$ and ($k=n$))
then

```

    {

        fp = cp + P[k];
        fw = Cw+W[k];

        for j=1 to k do X[j] = Y[j];
    }

}

```

```

if(Bound(cp,cw,k)≥ fp) then
    {
        y[k] = 0;
        if(k<n) then Bnap (K+1,cp,cw);

        if((cp>fp) and (k=n)) then
            {

                fp = cp;
                fw = cw;

                for j=1 to k do X[j] = Y[j];
            }

        }
    }
}

```

Algorithm for Bounding function:

Algorithm Bound(cp,cw,k)

// cp[→] current profit total.

//cw[→] current weight total.

//k[→]the index of the last removed item.

//m[→]the knapsack size.

```

{

    b=cp;
    c=cw;

    for I =- k+1 to n do
    {

        c= c+w[I];
        if (c<m) then b=b+p[I];

        else return b+ (1-(c-m)/W[I]) * P[I];
    }

    return b;
}

```

Example:

M= 6	Wi = 2,3,4	4 2 2
N= 3	Pi = 1,2,5	Pi/Wi (i.e.) 5 2 1
Xi = 1 0 1		

The maximum weight is 6

The Maximum profit is $(1*5) + (0*2) + (1*1)$

→5+1

→6.

Fp = (-1)

- $1 \leq 3$ & $0+4 \leq 6$

cw = 4, cp = 5, y(1) = 1

k = k+2

p 2 ≤ 3 but

$7 > 6$ so $y(2) =$

0

q So $\text{bound}(5, 4, 2, 6)$

B=5

C=4

I=3 to 3

C=6

$6 \neq 6$

So return $5 + (1 - (6 - 6)) / (2 * 1)$

□ 5.5 is not less

than fp. So, $k = k + 1$

(i.e.) 3.

$3 = 3$ & $4 + 2 \leq 6$

$cw = 6, cp = 6, y(3) = 1.$

K=4.

9. If $4 > 3$ then

$Fp = 6, fw = 6, k = 3, x(1) = 1 \ 0 \ 1$

The solution $X_i \rightarrow 1 \ 0 \ 1$

Profit $\rightarrow 6$

Weight $\rightarrow 6.$

BRANCH AND BOUND -- THE METHOD

The design technique known as **branch and bound** is very similar to backtracking (seen in unit 4) in that it searches a tree model of the solution space and is applicable to a wide variety of discrete combinatorial problems.

Each node in the combinatorial tree generated in the last Unit defines a *problem state*.

All paths from the root to other nodes define the **state space** of the problem.

Solution states are those problem states 's' for which the path from the root to 's' defines a tuple in the solution space. The leaf nodes in the combinatorial tree are the solution states.

Answer states are those solution states 's' for which the path from the root to 's' defines a tuple that is a member of the set of solutions (i.e., it satisfies the implicit constraints) of the problem.

The tree organization of the solution space is referred to as the **state space tree**.

A node which has been generated and all of whose children have not yet been generated is called a **live node**.

The **live node** whose children are currently being generated is called the *E*-node (node being expanded).

A **dead node** is a generated node, which is not to be expanded further or all of whose children have been generated.

Bounding functions are used to kill live nodes without generating all their children.

Depth first node generation with bounding function is called backtracking. State generation methods in which the *E*-node remains the *E*-node until it is dead lead to **branch-and-bound method**.

The term branch-and-bound refers to all state space search methods in which all children of the *E*-node are generated before any other live node can become the *E*-node.

In branch-and-bound terminology breadth first search(BFS)- like state space search will be called FIFO (First In First Output) search as the list of live nodes is a first -in-first -out list(or queue).

A **D-search** (depth search) state space search will be called LIFO (Last In First Out) search, as the list of live nodes is a list-in-first-out list (or stack).

Bounding functions are used to help avoid the generation of sub trees that do not contain an answer node.

The branch-and-bound algorithms search a tree model of the solution space to get the

solution. However, this type of algorithms is oriented more toward optimization. An algorithm of this type specifies a real -valued cost function for each of the nodes that appear in the search tree.

Usually, the goal here is to find a configuration for which the cost function is minimized. The branch-and-bound algorithms are rarely simple. They tend to be quite complicated in many cases.

Bounding:

A branch -and-bound searches the state space tree using any search mechanism in which all the children of the E-node are generated before another node becomes the E-node.

e assume that each answer node x has a cost $c(x)$ associated with it and that a minimum-cost answer node is to be found. Three common search strategies are FIFO, LIFO, and LC.

A cost function $\hat{c}(x)$ such that $\hat{c}(x) \leq c(x)$ is used to provide lower bounds on solutions obtainable from any node x . If upper is an upper bound on the cost of a minimum-cost solution, then all live nodes x with $\hat{c}(x) > \text{upper}$ may be killed as all answer nodes reachable from x have cost $c(x) \geq \hat{c}(x) > \text{upper}$. The starting value for upper can be set to infinity.

Clearly, so long as the initial value for upper is no less than the cost of a minimum-cost answer node, the above rule to kill live nodes will not result in the killing of a live node that can reach a minimum-cost answer node. Each time a new answer is found, the value of upper can be updated.

TRAVELLING SALESMAN PROBLEM

INTRODUCTION:

It is algorithmic procedures similar to backtracking in which a new branch is chosen and is there (bound there) until new branch is choosing for advancing.

Travelling salesman is a minimization problem. It can be solved using Branch and bound by generating state space tree by calculating cost and upper bound.

Reduced row or column: A row or column is said to be reduced if it has atleast one element zero.

Reduced Matrix: A matrix is said to be reduced if all its row and columns are reduced.

Reducing a matrix: For reducing a matrix, we can reduce all its row and columns by subtracting a smallest value in that row and column from all its elements in the row or column.